Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Programa de Licenciatura en Ingeniería Electrónica



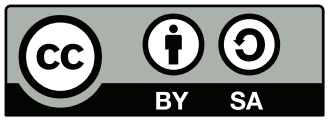
Sistema en dos etapas para la detección y reconocimiento de placas vehiculares usando modelos extremo a extremo y datos sintéticos

Informe de Trabajo Final de Graduación para optar por el título de

Ingeniero en Electrónica con el grado académico de Licenciatura

Justin Jaffeth Corea Masís

Cartago, 28 de noviembre, 2025



Este trabajo titulado *Sistema en dos etapas para la detección y reconocimiento de placas vehiculares usando modelos extremo a extremo y datos sintéticos* por Justin Jaffeth Corea Masís, se encuentra bajo la Licencia Creative Commons Atribución-ShareAlike 4.0 International.
Para ver una copia de esta Licencia, visite http://creativecommons.org/licenses/by-sa/4.0/.

©2025 Justin Jaffeth Corea Masís Instituto Tecnológico de Costa Rica

Declaro que el presente documento de tesis ha sido realizado enteramente por mi persona, utilizando y aplicando literatura referente al tema e introduciendo conocimientos y resultados experimentales propios.

En los casos en que he utilizado bibliografía he procedido a indicar las fuentes mediante las respectivas citas bibliográficas. En consecuencia, asumo la responsabilidad total por el trabajo de tesis realizado y por el contenido del presente documento.

Justin Jaffeth Corea Masís

Cartago, 28 de noviembre, 2025

Céd: 7-0292-0777

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería Electrónica
Trabajo Final de Graduación
Acta de Aprobación

Defensa de Trabajo Final de Graduación
Requisito para optar por el título de Ingeniero en Electrónica
Grado Académico de Licenciatura

El Tribunal Evaluador aprueba la defensa del trabajo final de graduación denominado *Sistema en dos etapas para la detección y reconocimiento de placas vehiculares usando modelos extremo a extremo y datos sintéticos*, realizado por el señor Justin Jaffeth Corea Masís y, hace constar que cumple con las normas establecidas por la Escuela de Ingeniería Electrónica del Instituto Tecnológico de Costa Rica.

Miembros del Tribunal Evaluador

Ing. Javier Mauricio Pérez Rodríguez, M.Sc
Profesor Lector

Dr. Sergio Alberto Arriola Valverde
Profesor Lector

Dr.-Ing. José Pablo Alvarado Moya
Profesor Asesor

Cartago, 28 de noviembre, 2025

Resumen

La presente tesis describe el desarrollo de un prototipo de sistema para el reconocimiento automático de números de placa (ANPR), pensado para su futura integración en un sistema de gestión inteligente de estacionamientos en el Campus Tecnológico Local San José.

El diseño del sistema ANPR plantea en dos etapas: detección de la placa vehicular y reconocimiento de caracteres. Para la detección se emplea la arquitectura YOLO, ajustada para detectar la región en la que se ubica la placa de los vehículos. Para el reconocimiento se utiliza Fast-Plate-OCR, adaptado al estilo tipográfico y variedad de placas costarricenses. Dado el tamaño limitado del conjunto de datos, se desarrolla un generador de imágenes sintéticas y un esquema de aumento de datos para entrenar el módulo de reconocimiento de caracteres. Finalmente, se realizaron experimentos para evaluar del desempeño tanto de cada componente del sistema por separado como del sistema integrado.

El sistema desarrollado alcanzó 98,4 % de precisión en el reconocimiento en placas vehiculares costarricenses y 93,5 % de precisión de la detección de la región donde se encuentra la placa vehicular. El sistema integrado procesa video a $3{,}84 \pm 0{,}79$ FPS, con una latencia media de $299 \pm 69{,}7$ ms.

Palabras clave: ANPR, IPLMS, extremo a extremo, imagen sintética ANPR, extremo a extremo, YOLOv11, IPLMS, Sintético

Abstract

This thesis describes the development of a prototype system for automatic license plate recognition (ANPR), designed for future integration into an intelligent parking management system at the San José Local Technology Campus.

The ANPR system design consists of two stages: vehicle license plate detection and character recognition. The YOLO architecture is used for detection, adjusted to detect the region where the vehicle license plate is located. Fast-Plate-OCR is used for recognition, adapted to the typographic style and variety of Costa Rican license plates. Given the limited size of the dataset, a synthetic image generator and a data augmentation scheme were developed to train the character recognition module. Finally, experiments were conducted to evaluate the performance of each component of the system separately and of the integrated system.

The developed system achieved 98.4 % accuracy in recognizing Costa Rican license plates and 93.5 % accuracy in detecting the region where the license plate is located. The integrated system processes video at $3{,}84 \pm 0{,}79$ FPS, with an average latency of $299 \pm 69{,}7$ ms.

Keywords: ANPR, IPLMS, end-to-end, synthetic images

a mis queridos padres

Agradecimientos

El resultado de este trabajo no hubiese sido posible sin el apoyo de mi profesor asesor Pablo Alvarado Moya, quien me orientó y brindó consejo a lo largo del desarrollo y presentación de los aspectos técnicos presentes en proyecto.

Por otro lado, agradezco profundamente a Paola Vega Castillo por haberme confiado la realización de este proyecto y por su gentil atención a las necesidades que surgieron durante la elaboración del trabajo. Asimismo, agradezco al personal del Campus Tecnológico Local San José por su hospitalidad y acompañamiento durante las visitas realizadas.

Justin Jaffeth Corea Masís

Cartago, 20 de noviembre de 2025

Índice general

Índice de figuras III

Índice de tablas V

1. Introducción 1
- 1.1. Gestión actual de parqueos en el Campus Tecnológico Local San José . . . 1
- 1.2. Experiencias previas de control vehicular en el TEC . . . 3
- 1.3. Retos en el control de acceso y disponibilidad de parqueos en el Campus San José . . . 4
- 1.4. Propuesta de un sistema de detección y reconocimiento de placas vehiculares 5
- 1.5. Objetivos del proyecto y organización del documento . . . 6

2. Marco teórico 7
- 2.1. Metodología clásica de detección y reconocimiento de números de placa . . 7
- 2.2. Detección de objetos . . . 10
  - 2.2.1. Tipos de arquitecturas populares . . . 10
- 2.3. Detección de matrículas mediante YOLO . . . 11
- 2.4. Reconocimiento de caracteres . . . 14
- 2.5. Compact Convolutional Transformers (CCT) . . . 15
  - 2.5.1. Motivación y visión general . . . 15
  - 2.5.2. De ViT a CCT: evolución de arquitecturas . . . 15
  - 2.5.3. Arquitectura y funcionamiento del CCT . . . 16
- 2.6. El efecto del desempeño de los hiperparámetros en el aprendizaje automático 17
- 2.7. Métricas de evaluación . . . 18

3. Implementación del sistema de detección y reconocimiento de placas 19
- 3.1. Generación sintética de imágenes . . . 19
- 3.2. Detección y extracción de ubicación de la placa en video e imágenes . . . 23
  - 3.2.1. Entrenamiento y evaluación del modelo . . . 24
- 3.3. Reconocimiento de número de placa . . . 25
- 3.4. Integración de la solución . . . 28

4. Resultados y análisis 30
- 4.1. Comparación de desempeño de detección de placas . . . 30
  - 4.1.1. Validación única (train/val split) . . . 30

4.1.2. Validación cruzada (k-fold) 35
4.1.3. Análisis de fallos en el mejor modelo de detección de placas 36
4.1.4. Evaluación respecto a modelo base 36
4.2. Características de las placas sintéticas generadas 40
4.3. Comparación de desempeño en reconocimiento de placas 41
4.3.1. Primera ronda de experimentos: exploración de hiperparámetros . . 41
4.3.2. Segunda ronda de experimentos: optimización de aumentos de datos 45
4.3.3. Análisis de fallos del OCR 48
4.3.4. Comparación respecto a modelo base 50
4.4. Evaluación del desempeño del sistema integrado 51

5. Conclusiones **53**
5.1. Recomendaciones 54

Bibliografía **55**

Índice de figuras

1.1. Ubicación de los parqueos obtenida mediante Google Maps. 2
1.2. Croquis del Campus Tecnológico Local San José [1]. 3
1.3. Diagrama de flujo del sistema ANPR 6

2.1. Técnica clásica de reconocimiento de números de placa (ANPR). 7
2.2. Ejemplos de placas vehiculares capturadas en ambiente iluminado a la derecha y la imagen binarizada por Otsu a la izquierda (tomado de [4]). . . . 7
2.3. Ejemplo de aplicación de transformada de Hough para detección de placa y rotación (tomado de [8]) 8
2.4. Segmentación de caracteres mediante CCA (tomado de [6]). 8
2.5. Clasificación alfanumérica con MLP (tomado de [9]). 9
2.6. Ejemplo de detección de placa (tomado de [14]). 11
2.7. Arquitectura original del detector de objetos YOLO (tomado de [13]). . . 13
2.8. Evolución de arquitecturas basadas en Transformers: ViT, CVT y CCT (tomado de [17]). 16

3.1. Diagrama de flujo para la generación de múltiples imágenes sintéticas. . . . 21
3.2. Generación de imagen de placa sintética. 22
3.3. Comparación de versiones de YOLO (tomado de [22]). 23
3.4. Entrenamiento del modelo de detección de placas. 24
3.5. Rutina de entrenamiento del OCR. 27
3.6. Diagrama de flujo de funcionamiento de sistema integrado 29

4.4. Ejemplos de falsos positivos de detección de placas 37
4.5. Ejemplos de falsos negativos de detección de placas 38
4.6. Ejemplo de falsos positivos completo 38
4.7. Ejemplo de detección de placas en imágenes 39
4.8. Ejemplos de placas sintéticas 40
4.9. Ejemplos de aumentado de datos 40
4.10. Métricas de aprendizaje en primer conjunto de experimentos 42
4.11. Comparación de hiperparámetros contra precisión en validación 43
4.12. Métricas de entrenamiento y validación de aumentado de placas con aumentado de datos 46
4.13. Casos de fallo en el reconocimiento de placas 49
4.14. Ejemplo de reconocimiento de placas. 50

4.15. Ejemplo de funcionamiento del sistema 52

Índice de tablas

1.1. Número de espacios por estacionamiento. 1

3.1. Posibles combinaciones de caracteres en placas costarricenses. L corresponde a una letra y la N a un dígito numérico (adaptado de [21]). 20
3.2. Características de placas costarricenses. 20
3.3. Comparación de complejidad entre versiones de YOLO11, usando imágenes de entrada de 640×640 píxeles, y T4 TRT10 es latencia en una T4 usando TensorRT 10 (tomado de [22]). 23
3.4. Criterios de evaluación para la selección del modelo OCR 25
3.5. Comparación ponderada de modelos OCR para reconocimiento de placas . 25
3.6. Comparación de desempeño base del OCR (tomado de [17]). 26

4.1. Resultados de los experimentos con YOLOv11 para detección de placas con validación única. 30
4.2. Resultados promedio ($\pm$ desviación estándar) de los experimentos con YOLO para detección de placas con validación cruzada $k = 5$. 35
4.3. Comparación de desempeño entre el modelo base y el modelo YOLO propuesto. 36
4.4. Rangos de hiperparámetros explorados en la primera ronda de experimentos. 41
4.5. Mejor conjunto de hiperparámetros de entrada, optimizador y modelo. . . . 44
4.6. Configuración del sistema utilizado para la ejecución de los experimentos de entrenamiento. 44
4.7. Resultados de duración de los experimentos de entrenamiento 45
4.8. Comparación de precisión entre el modelo *Baseline* y las variantes con diferentes transformaciones de datos. 47
4.9. Comparación de desempeño entre el modelo base y el modelo OCR propuesto. 50
4.10. Configuración del sistema utilizado para las pruebas de desempeño. 51
4.11. Resultados de desempeño del sistema integrado (YOLO + Fast-Plate-OCR). 51

Capítulo 1

Introducción

1.1. Gestión actual de parqueos en el Campus Tecnológico Local San José

En el Campus Tecnológico Local San José hay tres estacionamientos de automóviles, los cuales pueden ser usados ya sea por profesores, estudiantes o visitantes temporales. En el estacionamiento interno hay espacio para 12 vehículos y 5 motocicletas. En el estacionamiento ubicado en Casa Pacheco, ubicada al costado norte de la cuadra principal del Campus, sobre avenida 11, entre calles 5 y 7, hay espacio para 10 vehículos, y no hay espacio para motocicletas. Dado que el espacio continuó siendo insuficiente, la institución decidió alquilar el estacionamiento City, ubicado al costado sur del Campus sobre calle 5, el cual consta de 24 espacios para automóviles y 2 motocicletas. La Fig. 1.1, muestra la ubicación del Campus según Google Maps.

Ubicación	**Automóviles**	**Motocicletas**
Estacionamiento Interno	12	5
Casa Pacheco	10	0
Estacionamiento City	24	2
Total	**46**	**7**

Tabla 1.1: Número de espacios por estacionamiento.

En cada uno de los estacionamientos hay personal de seguridad que se encarga de la seguridad en el espacio. La entrada al estacionamiento interno se encuentra en el extremo opuesto de la entrada principal, donde se encuentra ubicada la caseta del guarda de seguridad. Para ingresar, los conductores deben primero detenerse frente a la caseta, notificar al guarda de seguridad y luego rodear la cuadra hasta llegar al portón de acceso al parqueo. Si no hay espacio disponible, se permite utilizar el estacionamiento de Casa Pacheco. En ese caso, es necesario consultar con un guarda ubicado dentro del edificio si

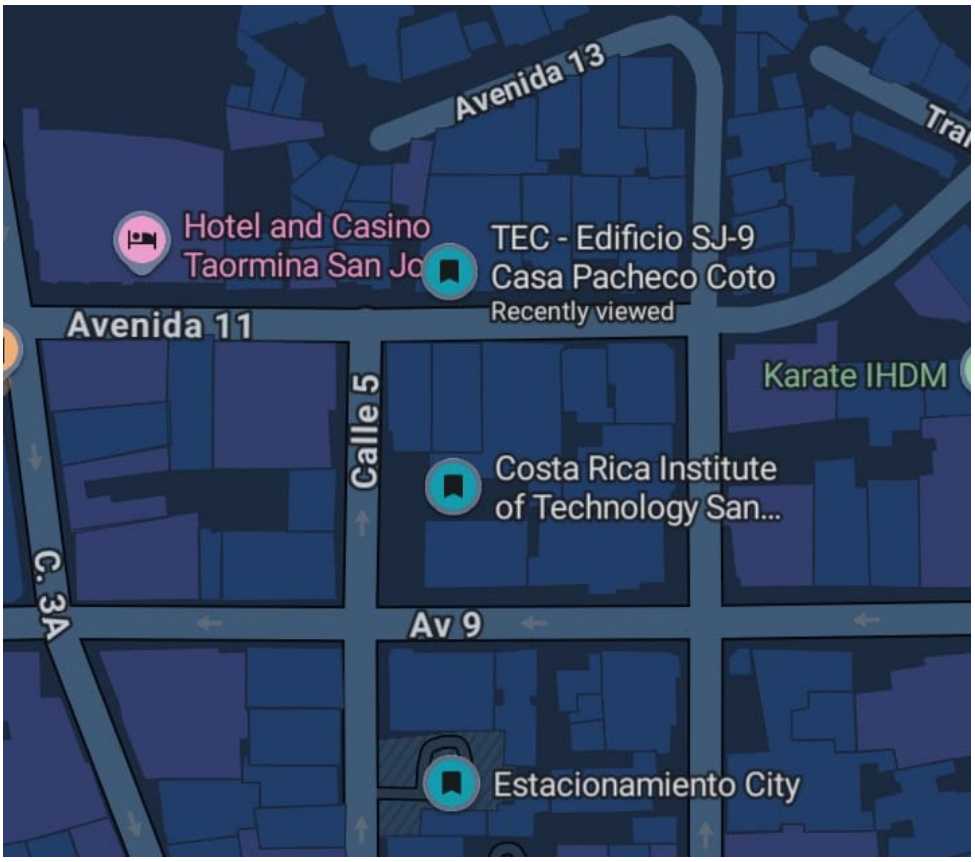


Figura 1.1: Ubicación de los parqueos obtenida mediante Google Maps.

hay cupo. Si la respuesta es afirmativa, el guarda de seguridad abre el portón manualmente para permitir el ingreso del vehículo. Dado lo reducido del espacio, en algunas ocasiones el guarda debe asistir a los conductores para maniobrar y estacionar adecuadamente. Como última opción está el estacionamiento City. En cuanto a este estacionamiento, también es necesario preguntar al personal de seguridad si hay espacios disponibles. Si los hay, se abre el portón para que los vehículos puedan ingresar. Cabe señalar que solo el portón del estacionamiento interno cuenta con un semáforo que indica si hay espacios disponibles. Este semáforo es operado manualmente por los guardas de seguridad.

Actualmente, no existe una coordinación entre los distintos guardas para la gestión conjunta de los estacionamientos. Además, las cámaras de seguridad instaladas en cada parqueo no pueden ser observadas por estos guardas, ya que el monitoreo está a cargo de un grupo distinto ubicado en Casa Rosada, dentro del campus principal (ver Figura 1.2). Dicho grupo y los guardas de los parqueos operan de forma independiente, pues pertenecen a contratos diferentes.

En cada uno de los estacionamientos existe una cámara de seguridad. En el estacionamiento interno, esta cámara está observando la entrada principal en una vista elevada, y también tiene una vista elevada del estacionamiento. En Casa Pacheco hay una única cámara que solo apunta a la entrada. Está en una pared al fondo y permite observar parcialmente la ubicación de los automóviles según lo reportado por el equipo de monitoreo.

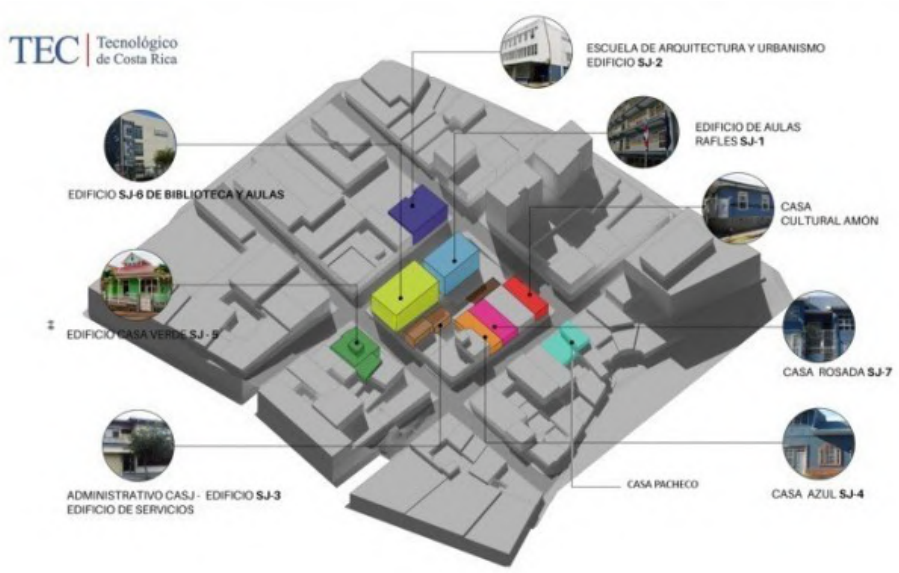


Figura 1.2: Croquis del Campus Tecnológico Local San José [1].

Sin embargo, esta cámara por el momento está inactiva a espera de mantenimiento. En el estacionamiento City hay un cámara con vista elevada desde la esquina suroeste. Da una vista desde arriba de todos los automóviles en el estacionamiento. La institución solo tiene acceso a la cámara del estacionamiento interno. A las cámaras de Casa Pacheco y Estacionamiento City solo tiene acceso monitoreo en Casa Rosada. Los guardas de seguridad de cada estacionamiento no tienen acceso a ninguna de las cámaras.

En síntesis, la gestión actual de los estacionamientos del Campus San José depende en gran medida de procedimientos manuales y de la coordinación verbal entre el personal de seguridad y los usuarios. Si bien esta situación puede atenderse parcialmente con mejoras administrativas, resulta evidente que también requiere una componente técnica que provea información confiable y en tiempo real sobre la disponibilidad de espacios y el flujo vehicular. Contar con dicha información permitiría a los guardas y a las personas usuarias tomar decisiones oportunas, agilizando la movilidad en la institución. En este contexto, los sistemas de reconocimiento automático de placas (ANPR) surgen como una alternativa tecnológica capaz de aportar esa componente informativa, y cuya implementación ha sido explorada previamente en el Tecnológico de Costa Rica.

1.2. Experiencias previas de control vehicular en el TEC

En un primer intento por abordar la problemática de gestión vehicular, y en coordinación con el Campus Tecnológico Local San José, se inició la implementación de un sistema de reconocimiento automático de placas vehiculares (ANPR) como parte de un sistema más amplio de gestión inteligente de parqueos (IPLMS). Este proyecto inicial buscaba aprovechar los recursos existentes en la sede, como las cámaras de seguridad, con el objetivo de mejorar la coordinación y el uso eficiente de los espacios de estacionamiento.

Este proyecto es tangencial al proyecto *"Destinos Turísticos Inteligentes: Tecnologías de Información y Comunicación Aplicadas al Turismo"* coordinado por Paola Vega Castillo. El cual contempló, entres sus líneas de trabajo, el desarrollo de parqueos inteligentes que puedan ser integradas en los Campus del Instituto Tecnológico de Costa Rica (TEC).

El diseño original logró identificar parcialmente las placas, pero no alcanzó un reconocimiento óptimo de los caracteres alfanuméricos. En pruebas realizadas, se reportó una tasa de éxito aceptable (90 %) para un conjunto de imágenes de prueba de placas internacionales, entrenando el modelo con dichas imágenes. Sin embargo, esta tasa se redujo drásticamente a menos del 20 % al evaluar el modelo con un conjunto de imágenes de placas costarricenses [2].

Además, para la detección de placas se utilizó un conjunto de datos reducido, compuesto por aproximadamente 300 imágenes de automóviles costarricenses, divididas entre entrenamiento y validación. Debido al tamaño limitado del conjunto, el modelo presentó sobreajuste, afectando su desempeño en nuevas imágenes.

El sistema desarrollado anteriormente se apoyó en técnicas clásicas de visión por computadora para realizar la identificación. Estas técnicas poseen bajo rendimiento ante variabilidad de diseños de placas. Los vehículos ingresan rápidamente y no hay barreras que los detengan, lo que reduce el tiempo disponible para la captura de imágenes. Aunque se intentaron reutilizar las cámaras existentes, estas no pueden ser reubicadas por motivos de seguridad. Su ubicación actual ——en puntos elevados y con ángulos inclinados——, junto con condiciones de iluminación adversas (sol directo en las mañanas y escasa luz en las noches), dificulta la visibilidad y complica el funcionamiento del sistema ANPR. Por estos motivos, el proyecto anterior no se llegó a implementar.

1.3. Retos en el control de acceso y disponibilidad de parqueos en el Campus San José

El problema específico a tratar en este trabajo es la inexistencia de un sistema de reconocimiento automático de placas vehiculares (ANPR) lo suficientemente robusto para ser aplicado en el control de acceso y gestión de estacionamientos del Campus San José. Si bien existen soluciones comerciales ampliamente usadas en parqueos privados, estas son propietarias, costosas y poco adaptables a las condiciones particulares del TEC. Por ello, se plantea la necesidad de desarrollar un sistema ANPR libre, flexible y de bajo costo, que pueda ajustarse a las limitaciones técnicas y presupuestarias de la institución.

1.4. Propuesta de un sistema de detección y reconocimiento de placas vehiculares

En el marco del proyecto general de gestión inteligente de estacionamientos, este trabajo se concentra en el desarrollo de un subsistema específico: el reconocimiento automático de placas vehiculares (ANPR). Este componente constituye la base técnica sobre la cual se pueden construir soluciones más amplias de control y coordinación de parqueos, ya que permite identificar de forma automática a los vehículos que ingresan al campus. La propuesta consiste en un prototipo de ANPR implementado mediante técnicas de visión por computadora y aprendizaje automático, diseñado para operar con las condiciones particulares del Campus San José y sin depender de licencias propietarias.

Este trabajo propone el diseño de una etapa de reconocimiento automático de placas vehiculares (ANPR). El subsistema ANPR se implementa por medio de estrategias de extremo a extremo; que contrasta con estrategias clásicas.

Los sistemas modulares tradicionales o de visión por computadora clásica deben conocer de previo la cantidad de caracteres, son sensibles a condiciones ambientales (iluminación, ángulo de visión, deformaciones), y su desempeño está limitado por el "eslabón más débil" de la cadena, lo que dificulta su reajuste.

Un modelo de extremo a extremo mitiga estos problemas al integrar todo el proceso de reconocimiento en una única arquitectura de aprendizaje profundo. Sin embargo, esto requiere grandes volúmenes de datos para entrenamiento [3]. Para suplir estos datos, en este trabajo, se propone un generador de datos sintéticos de placas, que permite entrenar el modelo de reconocimiento de manera robusta.

La propuesta de solución se organiza en dos etapas principales: detección y reconocimiento, según los siguientes principios:

1. **Captura de imágenes:** Se emplean cámaras electrónicas instaladas en puntos estratégicos donde las placas sean visibles al ingreso de automóviles y motocicletas.
2. **Detección de placa:** Se utiliza un modelo de detección de objetos para localizar y extraer la región correspondiente a la matrícula. Existen múltiples modelos pre-entrenados que pueden adaptarse.
3. **Reconocimiento de placa:** A diferencia de los métodos basados en segmentación de caracteres, se emplea un modelo extremo a extremo que integra la extracción y reconocimiento en una sola red convolucional. Las redes se entrenan con datos sintéticos generados y posteriormente evaluadas con muestras reales.

La figura 1.3 muestra las etapas del proyecto.

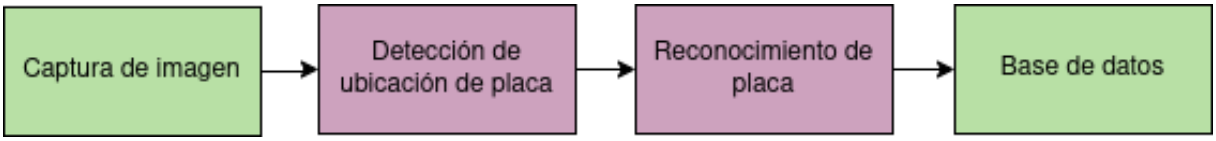


Figura 1.3: Diagrama de flujo del sistema ANPR

1.5. Objetivos del proyecto y organización del documento

El objetivo general del proyecto es desarrollar un prototipo de sistema automático para la detección y reconocimiento de caracteres en placas vehiculares costarricenses, aplicando técnicas de visión por computadora y aprendizaje automático, adaptado a las condiciones del Campus Local San José del Tecnológico de Costa Rica.

La tarea se divide en cuatro etapas:. La primera consiste en preparar un conjunto de datos adecuado para el entrenamiento del sistema, combinando imágenes reales de placas vehiculares costarricenses con datos sintéticos generados por software.

Al mismo tiempo, se implementan técnicas de aumento de datos en tiempo real durante el entrenamiento para robustecer el modelo ante condiciones adversas de iluminación, perspectiva de captura y resolución.

Posteriormente, se debe diseñar y entrenar una arquitectura de reconocimiento automático de placas basada en dos redes extremo a extremo, que incluya una etapa de detección de región de placa y otra de reconocimiento de texto sin segmentación manual.

Para finalizar, se evalúa el desempeño del sistema propuesto comparándolo contra un modelo base mediante métricas de reconocimiento y detección, con el fin de validar mejoras en precisión, exactitud completa y robustez.

Este documento se organiza de la siguiente manera: el capítulo 2 expone el marco teórico y el estado del arte en sistemas de reconocimiento de placas y gestión de estacionamientos inteligentes. A continuación, el capítulo 3 describe la metodología empleada para el desarrollo de la solución. Posteriormente, el capítulo 4 detalla los resultados obtenidos y el análisis de desempeño del sistema, así como las limitaciones encontradas. Para finalizar, el capítulo 5 presenta las conclusiones y recomendaciones para trabajos futuros.

Capítulo 2

Marco teórico

2.1. Metodología clásica de detección y reconocimiento de números de placa

El reconocimiento automático de números de placa (ANPR) involucra la extracción de información de una fuente de imagen o video. En los primeros sistemas de reconocimiento automático de matrículas, las técnicas clásicas de visión por computadora desempeñaban un papel fundamental para localizar y segmentar tanto la placa como sus caracteres. Típicamente, se dividía el procedimiento en cinco etapas como describe la figura 2.1.

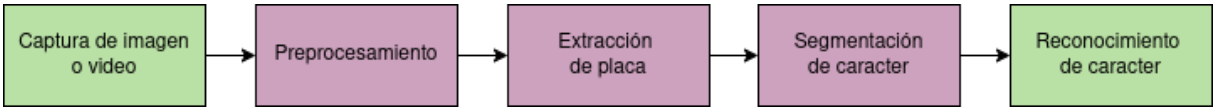


Figura 2.1: Técnica clásica de reconocimiento de números de placa (ANPR).

El preprocesamiento típico es binarización (conversión de imagen a escala de grises). Esto se realiza con el método Otsu [4]; en algunos casos se usaba imagen a color, pero estos casos tendían a tener menor precisión (ver figura 2.2). Luego se procedía a la extracción de la región que contenía placa. Entre los métodos más comunes para esta etapa destacan la Transformada de Hough [5] y el Análisis de Componentes Conectados (CCA) [6].



Figura 2.2: Ejemplos de placas vehiculares capturadas en ambiente iluminado a la derecha y la imagen binarizada por Otsu a la izquierda (tomado de [4]).

La Transformada de Hough es un procedimiento de extracción de características inicialmente concebido para la detección de líneas rectas (ver figura 2.3). Su principio consiste en transformar los puntos de borde en un espacio de parámetros, de modo que los puntos alineados en la imagen producen intersecciones en dicho espacio, permitiendo identificar líneas o formas geométricas definidas. En el contexto de ANPR, esta técnica resulta especialmente útil para detectar los contornos rectangulares de la matrícula [7].



(a) Detección de bordes de placa **(b)** Imagen rotada de la placa

Figura 2.3: Ejemplo de aplicación de transformada de Hough para detección de placa y rotación (tomado de [8])

Por su parte, el Análisis de Componentes Conectados (CCA), también conocido como extracción de blobs, permite etiquetar regiones de píxeles conectados dentro de una imagen binaria. Mediante un escaneo secuencial, se asigna una etiqueta única a cada conjunto de píxeles que comparten vecindad (4 u 8-conectividad). Esta técnica se utiliza tanto para la segmentación de la placa completa como para la separación de los caracteres individuales, ya que cada símbolo puede identificarse como un componente conectado [7].



Figura 2.4: Segmentación de caracteres mediante CCA (tomado de [6]).

En cuanto a la segmentación de caracteres. Uno de los más empleados es el de proyecciones

horizontales y verticales, en el que se calculan histogramas de densidad de píxeles para identificar los espacios vacíos que marcan la separación entre caracteres. Alternativamente, el mismo CCA aplicado a la imagen binarizada de la placa permite aislar cada carácter como un componente distinto (ver figura 2.4), aunque presenta limitaciones en casos de caracteres adheridos o solapados [6].

Finalmente, el reconocimiento de caracteres se realiza mediante redes neuronales artificiales (ANN), que actúan como clasificadores [7]. Por ejemplo, un perceptron multicapa (MLP) procesa la entrada mediante una capa inicial de neuronas, seguida de una o varias capas ocultas que aprenden asociaciones complejas, y finalmente una capa de salida que genera la decisión de clasificación (ver figura 2.5).

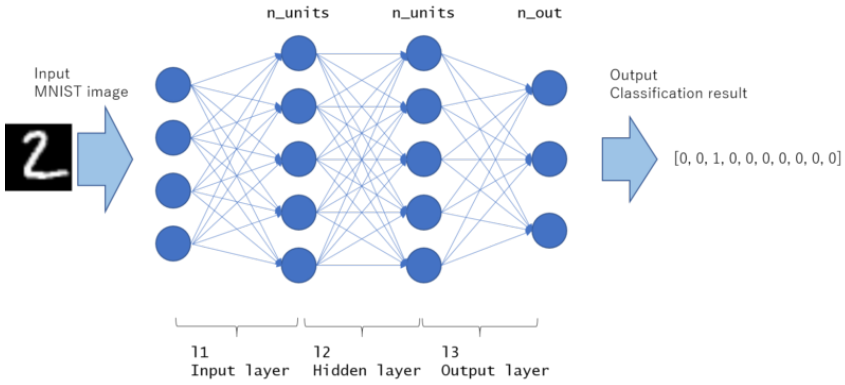


Figura 2.5: Clasificación alfanumérica con MLP (tomado de [9]).

Sin embargo, incluso con los mejores algoritmos, un ANPR puede requerir de hardware adicional para maximizar su precisión. La condición física de la placa, formatos no estandarizados del diseño de placas, la calidad de la cámara, iluminación, distorsión por movimiento, velocidad de procesamiento y memoria son algunos de los aspectos que afectan el rendimiento [10]. Los sistemas clásicos son especialmente susceptibles a estas limitaciones. En adición, en los sistemas clásicos el error en cada etapa se propaga a la siguiente, incrementando el error global y reduciendo su desempeño al eslabón más débil de la cadena.

Como respuesta a estas limitaciones, los sistemas modernos se apoyan en el aprendizaje automático mediante redes neuronales profundas, que aprenden directamente las características relevantes de las placas y pueden adaptarse mejor a las variaciones de diseño o captura. Este aprendizaje puede implementarse de dos maneras principales: mediante una red unificada que realiza simultáneamente la detección y el reconocimiento, o mediante un esquema en dos etapas, donde una primera red se encarga de la detección de las placas en la imagen, y una segunda red toma como entrada la región recortada para extraer la secuencia de caracteres.

Antes de describir las técnicas concretas, conviene explicar por qué se adopta esta separación lógica entre *detección* y *reconocimiento*. La detección localiza en la imagen las regiones que potencialmente contienen una placa (dibujando recuadros alrededor de los objetos de interés) y filtra el contenido irrelevante como el fondo o el resto del vehículo. El

reconocimiento, por su parte, toma esas regiones recortadas y extrae la información alfanumérica presente en ellas. Esta división permite optimizar cada módulo para una tarea específica (localización robusta frente a lectura precisa) y reduce el cómputo al procesar únicamente las regiones relevantes.

2.2. Detección de objetos

La *detección de objetos* consiste en la tarea de identificar y localizar elementos dentro de una imagen. En términos generales, el objetivo es determinar qué objetos aparecen y en qué lugar se encuentran. Para ello, el modelo analiza la imagen y estima, para cada objeto detectado, la clase a la que pertenece, las coordenadas en el plano de la imagen que definen una caja delimitadora (*bounding box*) que encierra dicho objeto, y una medida de confianza que indica la probabilidad de que el objeto pertenezca efectivamente a la clase asignada.

Una *clase* puede entenderse como una categoría abstracta que agrupa a todos los objetos con características y estructura similares. Por ejemplo, las clases "persona", "vehículo" o "placa" representan distintos tipos de elementos que pueden aparecer en una imagen, cada uno definido por patrones visuales distintivos.

En términos sencillos, la detección responde simultáneamente a las preguntas "¿qué hay en la imagen?" —determinando la clase— y "¿dónde está?" —determinando su posición mediante la caja delimitadora—. El resultado de un detector suele consistir en varias cajas superpuestas sobre la imagen, cada una con su etiqueta y un valor de confianza.

2.2.1. Tipos de arquitecturas populares

Se resumen a continuación, de forma introductoria, algunas de las arquitecturas más relevantes para la detección de objetos:

R-CNN (Region-based CNN). Introdujo la idea de proponer regiones candidatas en la imagen y procesarlas con una red convolucional para clasificar y ajustar cajas. Es conceptualmente simple y muy influyente, pero originalmente era costosa en tiempo porque procesaba cada propuesta por separado [11].

Faster R-CNN. Mejora a R-CNN integrando la generación de propuestas en la propia red (Region Proposal Network). Sigue siendo un enfoque en dos pasos (proponer regiones, luego refinar/clasificar), y suele ofrecer buena precisión, especialmente en escenarios donde la exactitud de la caja es importante. En general prioriza precisión frente a velocidad [12].

YOLO (You Only Look Once). YOLO es una familiar de modelos que proponen tratar la detección como un único problema de regresión: la imagen se procesa una sola vez y la red predice directamente las cajas y las probabilidades de clase. Esto lo hace muy rápido y adecuado para aplicaciones en tiempo real. A costa de eso, versiones muy simples pueden sacrificar algo de precisión en casos muy pequeños o densos, aunque las versiones modernas balancean muy bien velocidad y exactitud [13].

2.3. Detección de matrículas mediante YOLO

En el contexto del reconocimiento automático de matrículas (ANPR), la detección constituye la primera etapa del sistema, encargada de localizar en la imagen la región donde se encuentra la placa vehicular. Uno de los enfoques más extendidos y eficaces para esta tarea es la ya mencionada familia de modelos YOLO (You Only Look Once). Estos detectores procesan la imagen completa de una sola vez y generan, como salida, un conjunto de cajas delimitadoras con sus respectivas etiquetas y niveles de confianza. Durante el entrenamiento, la red recibe imágenes con las placas anotadas mediante cajas, aprendiendo así a predecir su posición y clase; mientras que en la fase de inferencia, el modelo analiza una imagen nueva y devuelve las regiones donde considera que hay una placa. Posteriormente, las detecciones con baja confianza se descartan mediante un paso de *filtrado*, de forma que sólo se conservan las regiones más probables para ser enviadas al módulo de reconocimiento (ver figura 2.6).

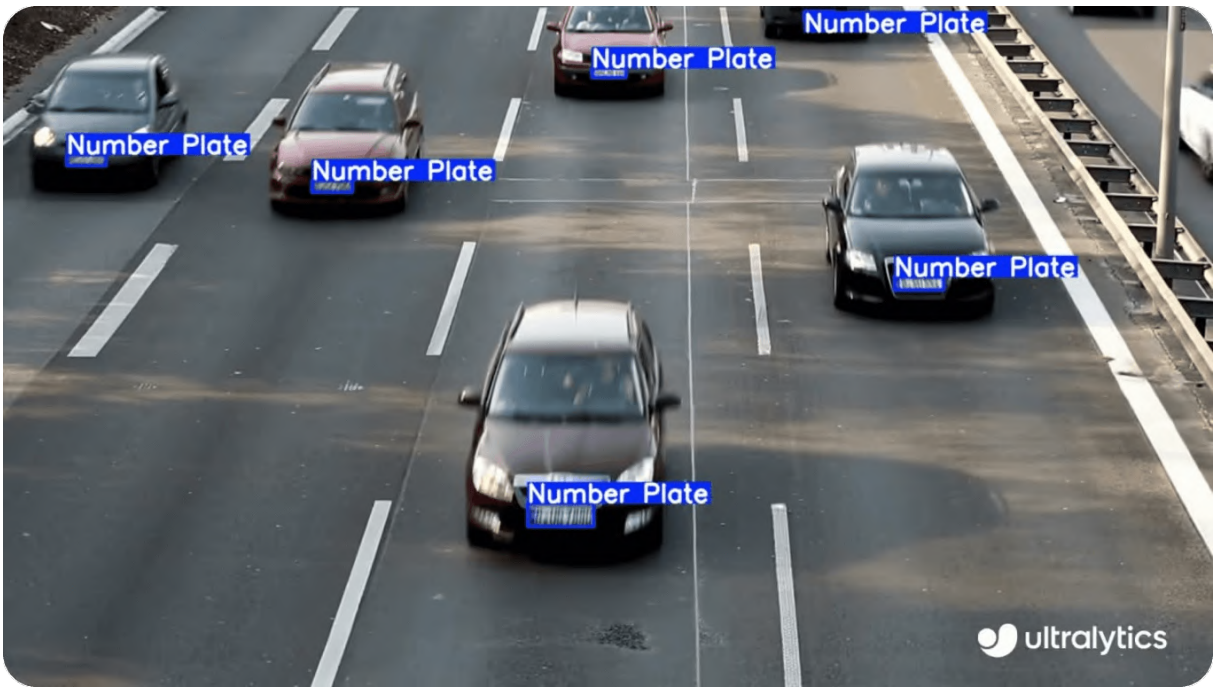


Figura 2.6: Ejemplo de detección de placa (tomado de [14]).

La rapidez y precisión que ofrece YOLO lo convierten en una herramienta idónea para aplicaciones en tiempo real, como la vigilancia de tráfico o el control de accesos. Su diseño

permite identificar la región de interés (la placa) bajo condiciones variables de iluminación, ángulo, movimiento o ruido en la imagen. Además, al procesar toda la imagen en un único paso, YOLO reduce el coste computacional frente a detectores de dos etapas (como R-CNN o Faster R-CNN), que primero generan propuestas de regiones y luego las clasifican. Este enfoque de una sola etapa le confiere una ventaja significativa en velocidad, aunque puede ser ligeramente más propenso a errores de localización fina. En la práctica, sin embargo, las versiones modernas de YOLO han alcanzado un equilibrio muy competitivo entre precisión y rendimiento, lo que las hace especialmente adecuadas para la detección de matrículas vehiculares.

YOLO realiza detección usando una única red neuronal. Usa características de la imagen completa para predecir las cajas delimitadoras. Realiza la detección para todas las clases definidas al mismo tiempo, lo que significa que YOLO "razona" globalmente sobre la imagen completa. Para ello divide la imagen en una cuadrícula $S \times S$. Si el centro de un objeto cae dentro de una celda de la cuadrícula, esa celda es responsable de detectar el objeto. Cada celda predice B cajas delimitantes y confianza para esas cajas. La confianza refleja qué tan seguro está el modelo que la caja contiene un objeto y qué tan seguro este está de la clase a la que corresponde lo que predice.

YOLO realiza la predicción de cajas delimitadoras y una puntuación asociada para cada caja. Cada predicción contiene 5 elementos, x, y, w, h y confianza. La coordenada (x, y) representa el centro de la caja relativo a los límites de la celda de la cuadrícula. El ancho w y alto h son relativos a la imagen completa. La confianza que devuelve la red es una puntuación que el modelo estima y cuyo objetivo es aproximar la probabilidad de que la caja contenga un objeto junto con la calidad del ajuste de la caja. En la formulación original de YOLO esta confianza se estima como $Pr(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}$; es decir, el producto entre la probabilidad de que haya un objeto en esa caja y la intersección sobre la unión (IoU) entre la caja predicha y la caja verdadera, siendo esta última la métrica utilizada para indicar la calidad de la estimación geométrica de la caja.

Hay que distinguir dos momentos: durante el entrenamiento la IoU se calcula explícitamente entre la predicción y la etiqueta (ese valor forma parte del "objetivo" para la componente de confianza en la función de pérdida); en cambio, en inferencia no es posible calcular la IoU frente a la etiqueta , porque la etiqueta no existe, en su lugar la red simplemente devuelve su estimación de confianza —es decir, una puntuación que pretende correlacionar con $Pr(\text{object}) \times IoU$ aunque no sea la IoU real calculable sin la etiqueta.

Adicionalmente, cada celda de la cuadrícula predice C probabilidades condicionales de clase $Pr(\text{Clase}_i \mid \text{Objeto})$. En la formulación clásica de YOLO se estima un único vector de probabilidades de clase por celda (independientemente del número de cajas B que predice la celda), de modo que la probabilidad conjunta para una caja y una clase puede expresarse aproximando $Pr(\text{Clase}_i,\text{Objeto}) \approx Pr(\text{Clase}_i \mid \text{Objeto}) \cdot \Pr(\text{Objeto})$, usando la confianza para aproximar $Pr(\text{Objeto})$.

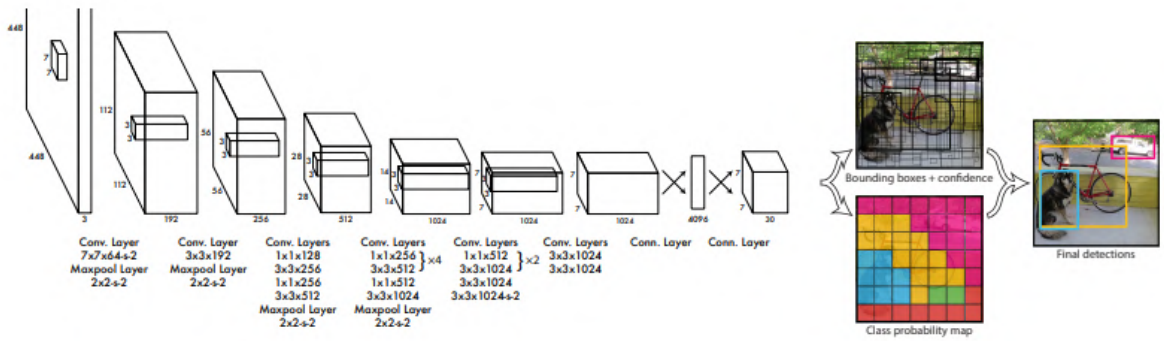


Figura 2.7: Arquitectura original del detector de objetos YOLO (tomado de [13]).

La arquitectura consta de varias capas convolucionales, seguida de unas pocas capas totalmente conectadas (ver figura 2.7). Estas capas normalmente se hallan preentrenadas en conjuntos de datos grandes, como PASCAL VOC e ImageNet 2012 hasta alcanzar 88 % de precisión en los mejores 5 modelos. A este se le agrega en el entrenamiento final una capa de reducción, seguido de capas convolucionales adicionales inicializadas con pesos aleatorios. A lo largo de sus versiones (YOLOv3, YOLOv4, YOLOv5, YOLOv7/8, y la más reciente YOLOv11), la arquitectura ha incorporado mejoras en la extracción de características. Con el tiempo, las versiones modernas han reemplazado estas capas por *backbones* (o red troncal) [1] más eficientes en hardware limitado y brindando un mejor balance entre precisión y rapidez [13].

Mejoras de YOLOv11

La versión más reciente de la familia, YOLOv11, introduce una serie de mejoras en su arquitectura respecto a la primera versión que buscan optimizar el equilibrio entre velocidad de procesamiento y precisión de detección [15]. Estas mejoras permiten que el modelo sea más eficiente sin perder capacidad de generalización, adaptándose tanto a entornos de cómputo limitados como a aplicaciones de alto rendimiento.

Entre las principales innovaciones se encuentran:

Bloque C3k2. Este bloque reemplaza partes de la estructura interna tradicional por una forma más ligera y eficiente de procesar las imágenes. Utiliza convoluciones de menor tamaño para reducir la carga computacional, logrando que el modelo procese más rápidamente sin afectar la calidad con que extrae las características visuales.

Módulo C2PSA. Incorpora un mecanismo de atención que permite al modelo concentrarse en las regiones más relevantes de la imagen. En términos simples, esto significa

[1] El *backbone* o red troncal es el elemento de la arquitectura del modelo encargado de la extracción automática de características de los datos de entrada, y es por lo general una red convolucional.

que la red aprende a "dar más importancia" a las zonas donde es más probable encontrar objetos, mejorando su capacidad para interpretar escenas complejas o con objetos parcialmente superpuestos.

SPPF (Spatial Pyramid Pooling – Fast). Este módulo realiza un análisis de la imagen a diferentes escalas de manera simultánea. Así, el modelo puede detectar con mayor precisión objetos grandes y pequeños dentro de una misma escena, sin necesidad de aumentar significativamente el tiempo de procesamiento. Esta característica es especialmente útil en contextos como el reconocimiento de placas, donde el tamaño y la posición del objeto pueden variar entre imágenes.

Versatilidad en tamaños de modelo. YOLOv11 ofrece distintas configuraciones, desde versiones muy ligeras (*nano*) hasta variantes de alto rendimiento (*extra-large*), lo que permite su implementación tanto en dispositivos embebidos con recursos limitados como en servidores dedicados. De esta manera, el modelo puede adaptarse a los requerimientos específicos del entorno de despliegue sin comprometer su desempeño.

En comparación con las versiones anteriores, YOLOv11 mejora su capacidad para capturar información a múltiples escalas y manejar distintos tamaños de objeto.

2.4. Reconocimiento de caracteres

Una vez localizada la región donde se encuentra la matrícula, el siguiente paso consiste en interpretar el contenido alfanumérico que aparece en ella. Este proceso se conoce como reconocimiento óptico de caracteres (*Optical Character Recognition*, OCR). En términos generales, el reconocimiento toma como entrada una imagen —en este caso, el recorte de la placa— y produce como salida una secuencia de texto, correspondiente a los caracteres presentes en la imagen.

El objetivo del OCR es traducir patrones visuales en símbolos legibles por una máquina. A diferencia de la detección, que se enfoca en localizar objetos, el reconocimiento requiere identificar con precisión la forma y estructura de cada carácter, incluso en condiciones difíciles de iluminación, inclinación, suciedad o distorsión. Tradicionalmente, esto se hacía segmentando la imagen carácter por carácter y aplicando clasificadores individuales. Sin embargo, los métodos modernos utilizan redes neuronales profundas capaces de procesar la secuencia completa de la imagen, aprendiendo directamente la correspondencia entre píxeles y texto sin necesidad de segmentar manualmente.

2.5. Compact Convolutional Transformers (CCT)

El reconocimiento de caracteres a partir de imágenes, como en el caso de las matrículas vehiculares, requiere modelos capaces de identificar patrones visuales que forman una secuencia coherente de símbolos. En este trabajo, dicha tarea se realiza mediante un modelo basado en transformers, una arquitectura que ha demostrado gran eficacia en tareas de secuencias, como el procesamiento del lenguaje natural y, más recientemente, en visión por computadora.

2.5.1. Motivación y visión general

Los *transformers* tradicionales se caracterizan por su mecanismo de *autoatención*, el cual permite al modelo determinar qué partes de una secuencia son más relevantes entre sí. En texto, estas unidades suelen ser palabras; en imágenes, se consideran pequeñas regiones o fragmentos. Esta capacidad de aprender dependencias globales hace que los Transformers sean herramientas muy poderosas, aunque a costa de un alto número de parámetros —entre 60 y 100 millones en el modelo original de [16]— y la necesidad de grandes volúmenes de datos para su entrenamiento.

En el contexto de reconocimiento de matrículas (ANPR), donde los conjuntos de datos suelen ser de tamaño moderado, estas exigencias hacen poco práctico el uso de Transformers convencionales. Por ello, surgen variantes más ligeras que combinan la eficiencia de las convoluciones con la capacidad de modelado contextual de la atención. Entre ellas se encuentra el Compact Convolutional Transformer (CCT), que equilibra precisión y eficiencia con un número mucho menor de parámetros (del orden de 0.3 a 3.8 millones en configuraciones reportadas) y un costo computacional reducido [17].

2.5.2. De ViT a CCT: evolución de arquitecturas

El Vision Transformer (ViT) fue la primera adaptación directa de los Transformers al dominio visual. ViT divide la imagen en fragmentos cuadrados no solapados (denominados *patches*) que se procesan como una secuencia de vectores [18]. Cada fragmento se convierte en una representación numérica (*embedding*) que resume su contenido visual, y el modelo aprende las relaciones entre ellos mediante autoatención. Este enfoque demostró gran potencial, pero presenta dos limitaciones: (1) la pérdida parcial de información espacial en los bordes entre fragmentos, y (2) su dependencia de grandes conjuntos de entrenamiento.

Para abordar estas limitaciones, surgió el Compact Vision Transformer (CVT), una versión reducida de ViT que agrupa las secuencias de fragmentos y reduce el número de parámetros. Posteriormente, al incorporar capas convolucionales en la etapa inicial de extracción de características, se obtiene el Compact Convolutional Transformer (CCT) [17]. Esta evolución —de ViT a CVT y luego a CCT— se ilustra en la figura 2.8.

2.5.3. Arquitectura y funcionamiento del CCT

A diferencia de ViT, que transforma directamente los fragmentos de imagen en vectores independientes, el CCT introduce un bloque convolucional inicial que extrae mapas de características antes de aplicar la atención. Este bloque, compuesto por convoluciones seguidas de activaciones ReLU y max-pooling, genera un mapa de características $x \in \mathbb{R}^{H \times W \times C}$, donde H y W representan las dimensiones espaciales y C el número de canales. De esta forma, el modelo conserva la información espacial local y mejora su robustez frente a inclinaciones, bordes difusos o ruido en las imágenes.

Posteriormente, la secuencia de características se procesa mediante bloques Transformer, que aplican autoatención para capturar relaciones entre distintas regiones de la imagen. Así, el modelo combina dependencias locales (aprendidas por las convoluciones) y globales (modeladas por la atención), obteniendo una representación visual más completa.

Para generar una representación final sin añadir parámetros adicionales, CCT emplea un mecanismo denominado Sequential Pooling (*SeqPool*). Este método pondera las salidas de cada posición mediante atención y las combina de forma adaptativa, priorizando las regiones más informativas de la imagen sin necesidad de añadir nuevas capas de clasificación.

Finalmente, la salida secuencial del modelo se alinea con la secuencia real de caracteres mediante la función de pérdida Connectionist Temporal Classification (CTC). La CTC

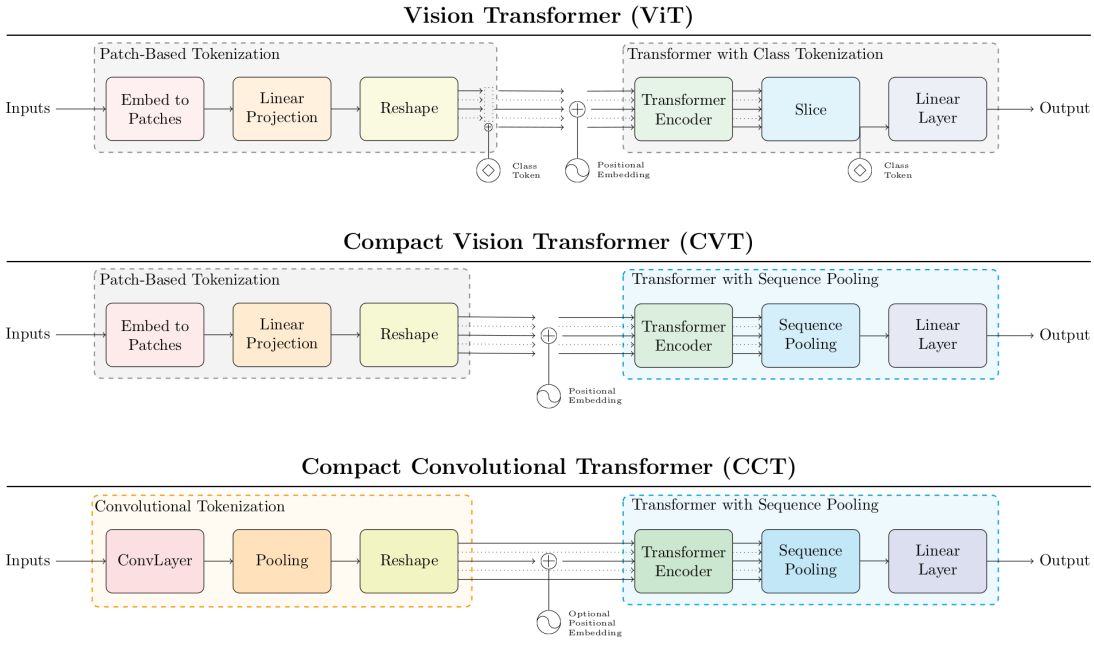


Figura 2.8: Evolución de arquitecturas basadas en Transformers: ViT, CVT y CCT (tomado de [17]).

permite entrenar el modelo sin segmentar manualmente cada carácter, ya que alinea dinámicamente las predicciones del modelo con la secuencia de texto esperada. Esto resulta especialmente útil en imágenes donde los caracteres están unidos, deformados o parcialmente cubiertos.

2.6. El efecto del desempeño de los hiperparámetros en el aprendizaje automático

El entrenamiento de modelos de aprendizaje profundo implica la definición de un conjunto de hiperparámetros, tales como la tasa de aprendizaje, el tamaño de lote, el número de capas, el factor de regularización o las políticas de aumento de datos, entro otros. A diferencia de los pesos internos de la red, que son aprendidos durante el entrenamiento, los hiperparámetros deben ser definidos previamente, y su configuración incide de manera directa en la capacidad de generalización del modelo [19].

En tareas como la detección y el reconocimiento automático de placas vehiculares, la correcta selección de hiperparámetros resulta crítica, especialmente por las limitaciones de datos disponibles y la alta variabilidad en las condiciones de captura. Una elección inadecuada puede llevar al sobreajuste (el modelo memoriza ejemplos de entrenamiento, pero falla en casos nuevos) o al subajuste (la red no alcanza a aprender patrones relevantes), comprometiendo la precisión del sistema.

En arquitecturas de detección como YOLO, hiperparámetros tales como la tasa de aprendizaje, el tamaño de las imágenes de entrada, el número de épocas de entrenamiento y las técnicas de aumento (por ejemplo, Mosaic o MixUp) afectan significativamente la precisión y la velocidad de inferencia. De manera análoga, en modelos de reconocimiento basados en Compact Convolutional Transformers (CCT), factores como la dimensión de representación vectorial, el número de capas y cabezas de atención, junto con regularizadores como descarte aleatorio o decaimiento de pesos, influyen en la estabilidad del entrenamiento y en la capacidad de trabajar con conjuntos de datos pequeños.

Por estas razones, la optimización de hiperparámetros constituye una etapa indispensable dentro del diseño del sistema propuesto. A través de estrategias de búsqueda sistemática o heurística, es posible identificar configuraciones que logren un balance entre precisión, velocidad y robustez, permitiendo que el sistema ANPR alcance un desempeño adecuado en condiciones reales.

En conclusión, los métodos clásicos de ANPR, basados en segmentación y reconocimiento secuencial, presentan limitaciones ante variaciones en condiciones de captura y diseños no estandarizados de placas. Los enfoques modernos basados en aprendizaje profundo, como la detección mediante YOLOv11 y el reconocimiento con Compact Convolutional Transformers (CCT), permiten superar estas restricciones, ofreciendo robustez frente a ruido, deformaciones y variaciones de iluminación. Estas arquitecturas, combinadas con una correcta optimización de hiperparámetros, constituyen la base tecnológica del siste-

ma propuesto, asegurando un equilibrio entre precisión, velocidad y adaptabilidad ante escenarios reales de aplicación.

2.7. Métricas de evaluación

La evaluación objetiva del rendimiento de un sistema de reconocimiento automático de placas vehiculares requiere métricas que reflejen tanto la calidad de la detección de matrículas como la precisión en el reconocimiento de caracteres. Entre las más empleadas destacan:

Detección de matrículas:

- **Precisión (Precision):** proporción de las detecciones que son correctas sobre el total de detecciones realizadas por el modelo. Un valor alto indica pocos falsos positivos.
- **Exhaustividad (Recall):** proporción de objetos correctamente detectados sobre el total de objetos presentes en la imagen. Un valor alto indica pocos falsos negativos.
- **mAP50 (mean Average Precision a IoU=0.5):** métrica estándar en visión por computadora para detección de objetos. Calcula el promedio de la precisión con distintos umbrales, considerando una detección como correcta si la superposición entre caja predicha y caja verdadera supera un 50 % de IoU (*Intersection over Union*). Resume el balance entre precisión y exhaustividad en una sola medida. Un valor alto de mAP indica que el modelo no solo detecta correctamente la presencia de la matrícula, sino que también predice con exactitud su localización

Reconocimiento de caracteres:

- **Precisión de longitud de caracteres:** porcentaje de placas en las que el sistema reconoce correctamente la cantidad de caracteres, independientemente de su contenido. Evalúa la consistencia estructural del reconocimiento.
- **Precisión de placas:** porcentaje de placas en las que la secuencia completa de caracteres fue reconocida correctamente. Esta métrica es estricta, ya que considera la matrícula como una unidad única. Esto es idéntico al concepto de WER —Word Error Rate— [20], pero aplicada directamente a la secuencia completa.

Capítulo 3

Implementación del sistema de detección y reconocimiento de placas

3.1. Generación sintética de imágenes

Los sistemas de extremo a extremo tienen la desventaja de requerir grandes cantidades de datos para conseguir resultados con bajo error de entrenamiento y generalización; habitualmente en el rango de 10 mil - 50 mil fotografías. En el tiempo disponible para la ejecución del proyecto no es posible capturar y etiquetar tal cantidad de imágenes, además el costo de etiquetar los datos es elevado. El problema se mitiga utilizando imágenes sintéticas; incrementando el conjunto de datos disponibles. Tales imágenes son creadas con base a la caracterización de las placas reales y la fuente tipográfica usada en las mismas.

El conjunto artificial es utilizado para entrenar el modelo, mientras que el conjunto de placas auténticas es usado para la evaluación del sistema. Con ello se puede evaluar, indirectamente, la utilidad de la estrategia, y la representatividad con la que conjunto sintético asemeja al conjunto real en sus características esenciales.

En Costa Rica a las placas vehiculares se les asigna un código según el tipo de vehículos. El reglamento sobre placas vehiculares [21] especifica los códigos existentes, así como el color del fondo de la placa y el color de los caracteres. Hay más de 100 tipos de placas. Automóviles con códigos atípicos, como el de placas extranjeras, son poco probables de ser evaluadas; además, en el conjunto de imágenes reales no hay muestras de todos los tipos de placas. Por este motivo solo se sintetizan placas basadas en las existentes en el conjunto de datos real y las demás se dejan como casos no cubiertos o casos extremos. Los casos a cubrir se muestran en la tabla 3.1, y sus características en la tabla 3.2.

Se generan los datos utilizando plantillas de placas reales y la fuente tipográfica real, realizando recortes de los caracteres y dejando plantillas lo más limpias posibles. Un programa diseñado para este propósito crea placas aleatorias de forma sistemática, utilizando el proceso mostrado en la figura 3.1.

Tipo	Combinación	Ejemplo
Particular	NNNNNN	
Particular	LLL-NNN	
TEC	265 | NN	
TEC	265 | NNN	
Carga ligera	CL | NNNNNN	
Discapacitado	D NNN	
Motocicleta	M NNNNNN	
Motocicleta	M $\frac{\text{NNN}}{\text{NNN}}$	

Tabla 3.1: Posibles combinaciones de caracteres en placas costarricenses. L corresponde a una letra y la N a un dígito numérico (adaptado de [21]).

Código	Color de placa	Color de caracteres	Ejemplo
(vacío)	Blanco	Azul	
(vacío)	Verde	Azul	
265	Blanco	Verde	
CL	Blanco	Rojo	
D	Blanco	Azul	
M	Blanco	Azul	

Tabla 3.2: Características de placas costarricenses.

Durante el entrenamiento del modelo de reconocimiento de placas, se aplican técnicas de aumento de datos (data augmentation) para incrementar la variabilidad de las imágenes y mejorar la capacidad de generalización del modelo. El tipo de aumento y la intensidad deben ser cuidadosamente seleccionados para evitar introducir artefactos que puedan confundir al modelo. Por lo tanto, se seleccionan posibles transformaciones e intensidades basadas en las características del conjunto de datos real, y luego durante el entrenamiento se realizan experimentos para determinar cuáles combinaciones producen mejores resultados. Entre las técnicas de aumento de datos utilizadas se incluyen:

- **Rotación (imagen):** Se aplican rotaciones aleatorias en el rango de $\pm 20°$ para simular inclinaciones de cámara o variaciones en el montaje de la placa.

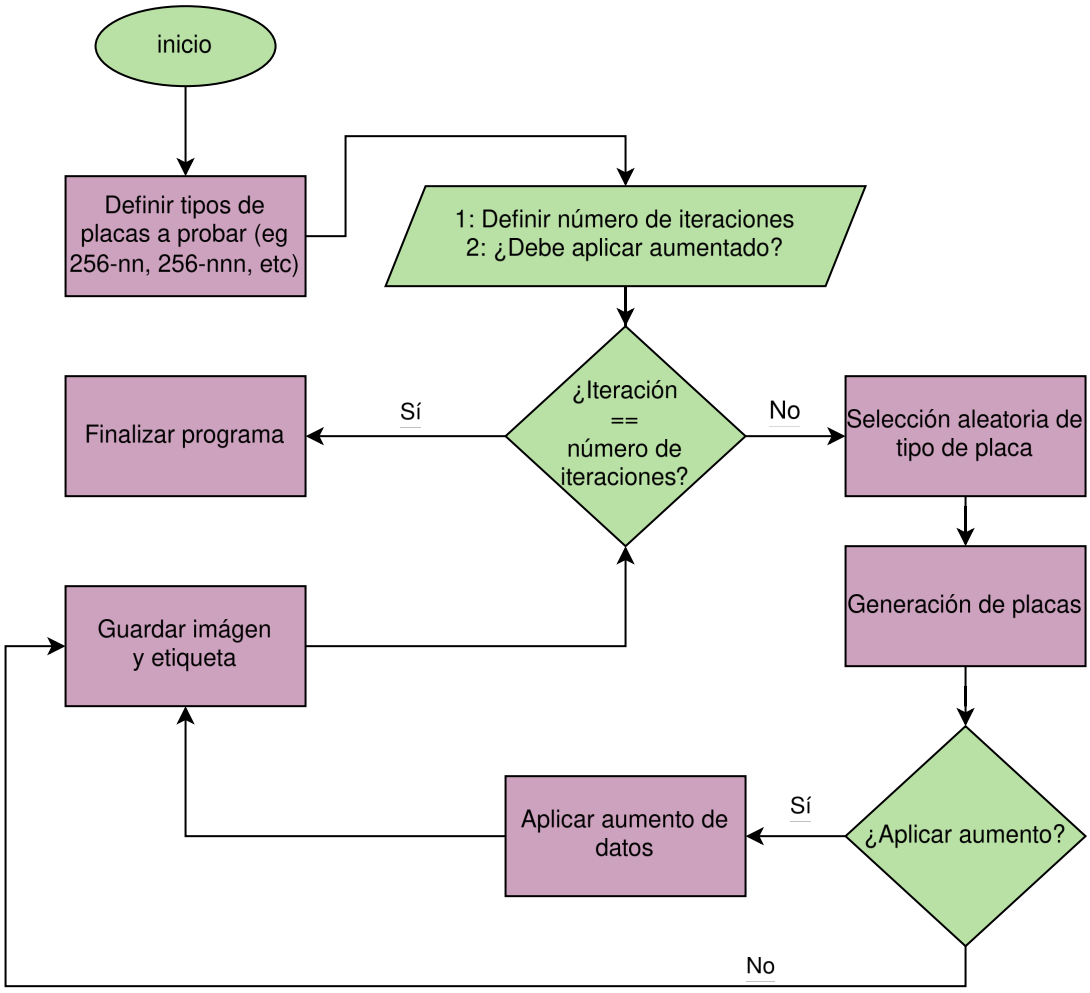


Figura 3.1: Diagrama de flujo para la generación de múltiples imágenes sintéticas.

- **Escalado (imagen):** Se aplican escalados aleatorios con un factor de 0,5 a 1,0 para representar cambios de distancia o zoom.
- **Transformación de perspectiva (imagen):** Se aplican transformaciones de perspectiva con una desviación σ entre 10 y 60, y desplazamientos relativos dx,$dy \in [0{,}1, 0{,}6]$ para simular distintos ángulos de captura.
- **Cambio de brillo y contraste (imagen):** Se varía el brillo y el contraste mediante factores en el rango $[0{,}7, 1{,}3]$, reproduciendo condiciones de iluminación natural.
- **Ruido (imagen):** Se añade ruido gaussiano con media 0 y varianza entre 0,001 y 0,01, imitando ruido del sensor o compresión digital.
- **Desenfoque (imagen):** Se aplica desenfoque gaussiano o de movimiento con un radio de 0,5 a 2 píxeles para representar movimiento o fuera de foco.
- **Pixelado (imagen):** Se simula pérdida de resolución aplicando pixelado con una escala de reducción entre 0,6 y 1,0 del tamaño original.

- **Desplazamiento RGB (imagen):** Se aplican desplazamientos de canales de color con límites de ± 20 unidades por canal (R, G, B), simulando aberraciones cromáticas.
- **Compresión JPEG (imagen):** Se simula degradación por compresión con calidades entre 35 y 75, reproduciendo artefactos comunes en imágenes de baja calidad.
- **Cambio de relación de aspecto (imagen):** Se modifica la relación de aspecto entre 0,5 y 1,0, generando distorsiones leves en ancho o alto.
- **Escala de grises parcial (imagen):** Se convierte la imagen a escala de grises con mezcla ponderada promedio, para aumentar la robustez ante pérdida de color.
- **Texturizado de caracteres (carácter):** Se aplica variación aleatoria del color base en un rango de ± 20 unidades por canal RGB, imitando pintura irregular o desgaste.
- **Degradado de caracteres (carácter):** Se aplica erosión y difuminado sobre bordes con kernels de 2×2 píxeles y entre 0 y 2 iteraciones, simulando desgaste del relieve o suciedad.

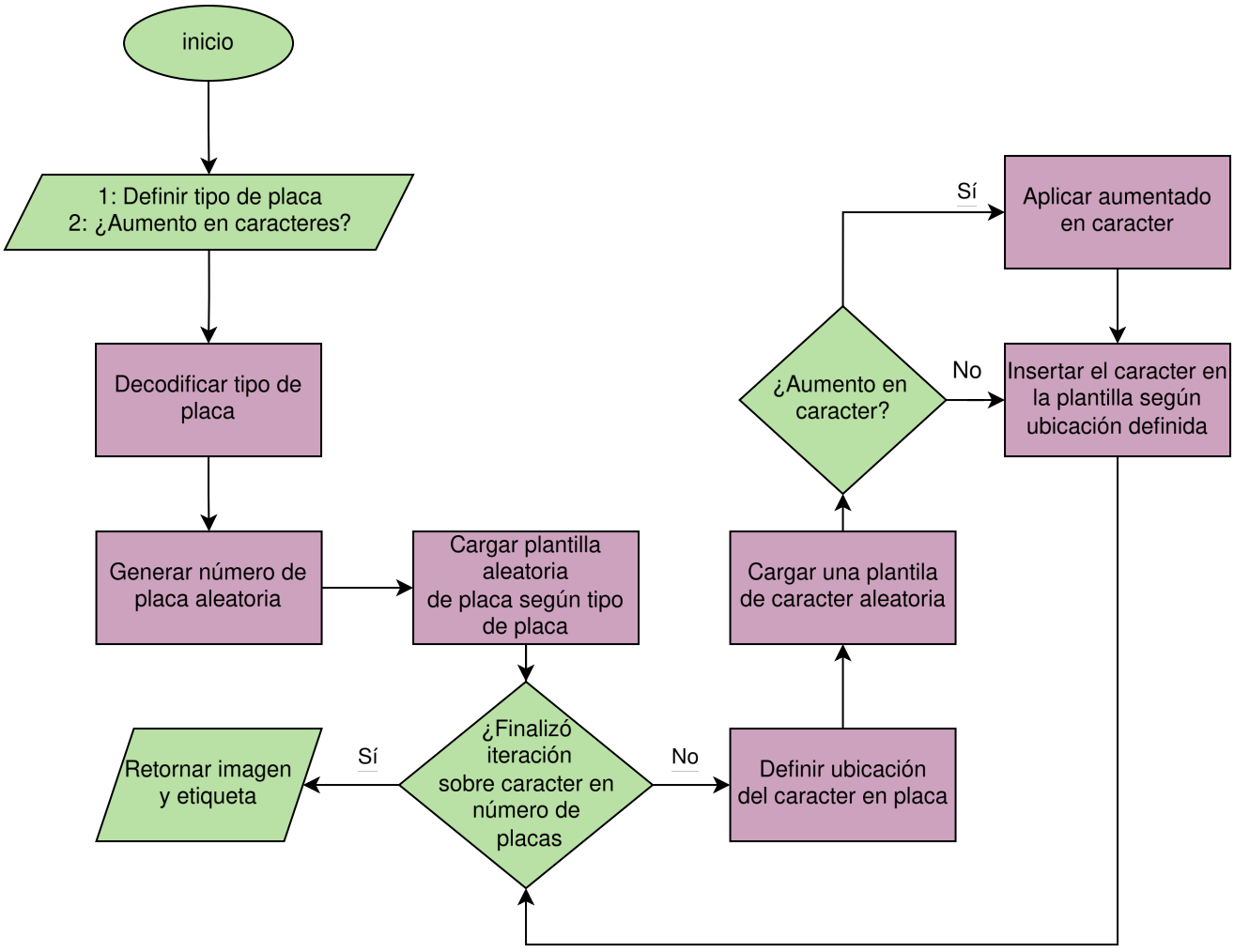


Figura 3.2: Generación de imagen de placa sintética.

- **Cambio de color por tipo (carácter):** Los caracteres se recolorean según su categoría (azul, rojo o verde), manteniendo coherencia con los tipos de placa reales.
- **Adición de tornillos u objetos (imagen):** Se añaden elementos visuales externos de tamaño relativo entre 5 % y 10 % del ancho de la placa, ubicados en posiciones aleatorias.

Puesto que se puede aplicar aumentado de datos en distintos elementos de la placa, en la rutina de generación de una única placa se especifican puntos de control para decidir de forma aleatoria qué aumento aplicar en cada caso (ver figura 3.2). Cabe destacar que la selección de la plantilla y el carácter es aleatorio, seleccionando un elemento cualquiera de un directorio que contiene diferentes representaciones de cada clase, pero se conoce en cada momento la correspondencia de la imagen con su etiqueta de número de placa.

3.2. Detección y extracción de ubicación de la placa en video e imágenes

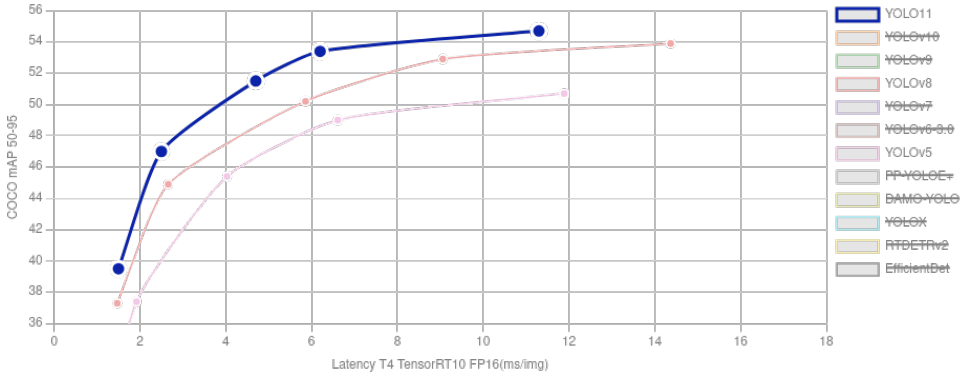


Figura 3.3: Comparación de versiones de YOLO (tomado de [22]).

Modelo	CPU (ms)	T4 TRT10 (ms)	mAP^{val}	Parámetros (M)	FLOPS (B)
YOLO11n	$56{,}1 \pm 0{,}8$	$1{,}5 \pm 0{,}0$	39.5	2.6	6.5
YOLO11s	$90{,}0 \pm 1{,}2$	$2{,}5 \pm 0{,}0$	47.0	9.4	21.5
YOLO11m	$183{,}2 \pm 2{,}0$	$4{,}7 \pm 0{,}1$	51.5	20.1	68.0
YOLO11l	$238{,}6 \pm 1{,}4$	$6{,}2 \pm 0{,}1$	53.4	25.3	86.9
YOLO11x	$462{,}8 \pm 6{,}7$	$11{,}3 \pm 0{,}2$	54.7	56.9	194.9

Tabla 3.3: Comparación de complejidad entre versiones de YOLO11, usando imágenes de entrada de 640×640 píxeles, y T4 TRT10 es latencia en una T4 usando TensorRT 10 (tomado de [22]).

La detección de placas vehiculares en imágenes y video se realiza aplicando ajuste fino (fine tunning) a modelos existentes. Se utiliza YOLOv11n, pues de las versiones de YOLO, la versión 11 ofrece un buen balance entre precisión y velocidad, es adecuada para aplicaciones en tiempo real, y es más precisa y veloz que versiones anteriores como YOLOv5 o YOLOv8 (ver figura 3.3). Por otro lado, la versión n es la más ligera, lo que facilita su implementación en dispositivos con recursos limitados (ver tabla 3.3). Las velocidades reportadas en CPU corresponden a inferencia sobre el modelo exportado a ONNX, mientras que la columna T4 TRT10 muestra la latencia en milisegundos en una GPU NVIDIA T4 usando TensorRT 10.

3.2.1. Entrenamiento y evaluación del modelo

Con la implementación de Ultralytics [23] se entrena el modelo. Dicha implementación incluye procedimientos de aumento de datos, así como las especificaciones sobre el tipo y formato de las etiquetas, por lo que el entrenamiento se simplifica a un único bloque, como se muestra en la figura 3.4.

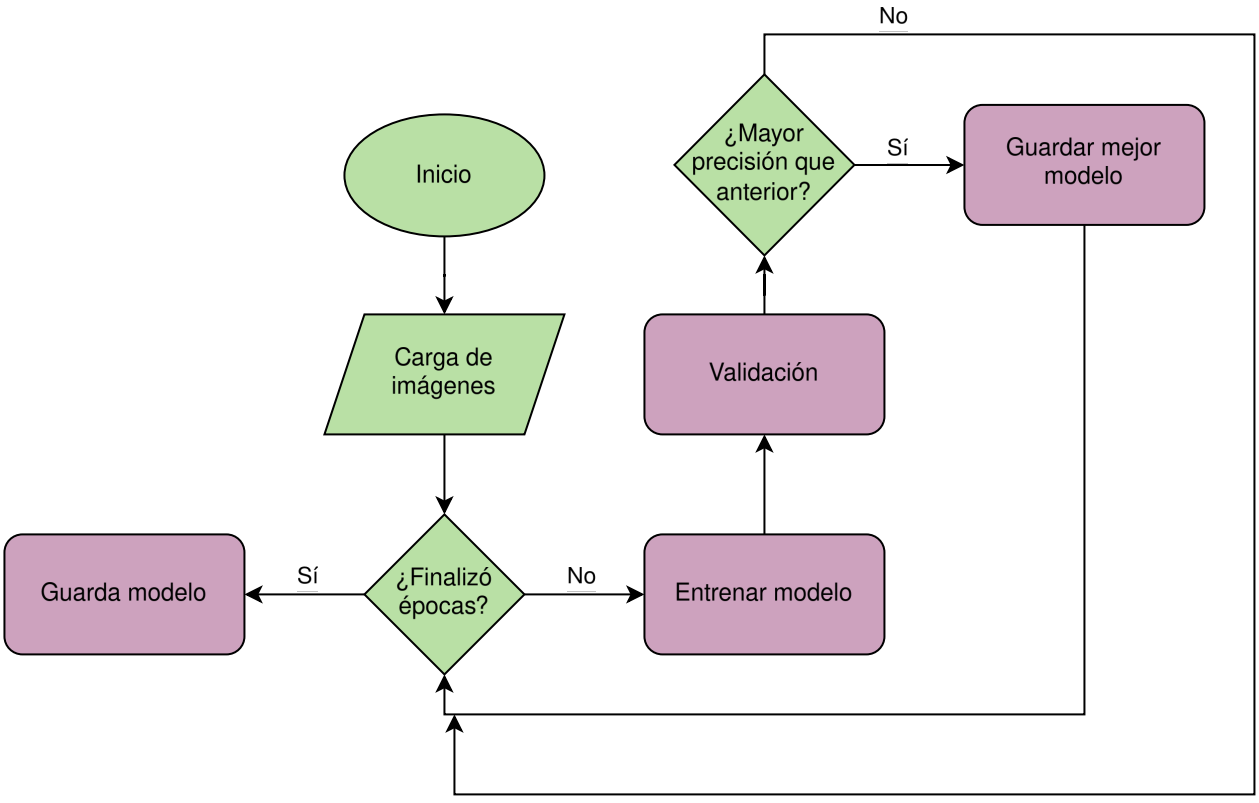


Figura 3.4: Entrenamiento del modelo de detección de placas.

3.3. Reconocimiento de número de placa

La etapa de reconocimiento de números de placas requiere seleccionar un modelo de OCR adecuado, que equilibre precisión, velocidad y facilidad de implementación. Para ello, se evaluaron cuatro alternativas principales: LPRNet, CLPRNet PaddleOCR y Fast-Plate-OCR. Cada una de estas arquitecturas presenta ventajas y limitaciones en términos de desempeño, compatibilidad con hardware y programas modernos, facilidad de entrenamiento y grado complejidad en el mantenimiento del proyecto.

Criterio	**Peso**	**Descripción**
Precisión de reconocimiento	3	Nivel de acierto en la predicción de caracteres.
Velocidad de inferencia	3	Rapidez promedio de procesamiento por imagen.
Facilidad de entrenamiento / ajuste fino	2	Complejidad del entrenamiento y disponibilidad de herramientas.
Compatibilidad con entornos actuales	2	Capacidad de ejecución con versiones recientes de PyTorch y CUDA.
Mantenimiento del proyecto	1	Frecuencia de actualización y soporte activo.

Tabla 3.4: Criterios de evaluación para la selección del modelo OCR

Criterio	**Peso**	**LPRNet**	**CLPRNet**	**PaddleOCR**	**Fast-Plate-OCR**
Precisión	3	3	5	4	**5**
Velocidad	3	4	3	3	**5**
Facilidad de entrenamiento	2	3	2	4	**5**
Compatibilidad	2	1	3	2	**5**
Mantenimiento	1	1	2	1	**3**
Puntaje Ponderado		29	33	33	**46**

Tabla 3.5: Comparación ponderada de modelos OCR para reconocimiento de placas

Con el fin de fundamentar la selección del modelo final, se elaboró una matriz de decisión (método de Pugh) considerando los criterios de evaluación mostrados en la tabla 3.4. A partir de estos criterios, se evaluaron las arquitecturas candidatas (tabla 3.5) y se calculó una puntuación ponderada según la suma de los productos entre calificación y peso.

De acuerdo con la comparación, Fast-Plate-OCR obtuvo la puntuación más alta, destacando por su ligereza, velocidad de procesamiento y facilidad de ajuste fino, además

de contar con un proyecto reciente y en constante actualización. Los modelos que ofrece son rápidos, eficientes y compatibles con las versiones actuales de PyTorch y GPU NVIDIA, pudiendo entrenarse tanto desde cero como mediante ajuste fino, lo que simplifica la integración dentro del flujo experimental del sistema.

En contraste, otras arquitecturas presentan limitaciones técnicas o de mantenimiento. Por ejemplo, LPRNet fue una de las primeras propuestas funcionales para reconocimiento de placas, pero su desarrollo data de hace aproximadamente seis años, y su código resulta difícil o incluso imposible de ejecutar en versiones recientes de bibliotecas [24]. Además, su arquitectura es más rígida y requiere modificaciones sustanciales para adaptar la entrada y salida a los formatos de datos utilizados en este proyecto.

Por otra parte, CLPRNet propone un enfoque extremo a extremo que unifica la detección y el reconocimiento de placas dentro de una sola red [25]. Si bien ofrece un alto potencial de precisión, su entrenamiento demanda un etiquetado detallado que incluye las coordenadas de los vértices de la placa y la clasificación por tipo de matrícula. Esto implicaría reetiquetar completamente el conjunto de datos disponible y redefinir la estrategia de generación sintética de imágenes, lo cual excede el alcance de este trabajo.

Finalmente, se consideró PaddleOCR, una herramienta ampliamente usada para OCR en contextos generales; sin embargo, al igual que LPRNet, su implementación se encuentra desactualizada y no puede ejecutarse de forma nativa en entornos recientes de PyTorch y CUDA sin reescribir parte del código fuente, lo que comprometería la reproducibilidad y portabilidad del sistema.

En síntesis, tras evaluar las distintas opciones, se decidió emplear Fast-Plate-OCR como modelo base para el reconocimiento de caracteres de placas.

Fast plate OCR cuenta con dos modelos pre entrenados. En pruebas de concepto inicial el modelo más pequeño (XS) resultó en un mejor compromiso entre precisión y velocidad de procesamiento, por lo que se escoge como modelo base para realizar los experimentos. El reporte de desempeño brindado por los desarrolladores del modelo base se muestra en la tabla 3.6, el cual fue evaluado en el hardware NVIDIA RTX 3090 GPU. La rutina de entrenamiento del modelo se observa en la figura 3.5.

Modelo	**Tamaño**	**Latencia (ms)**	**Placas/segundo**
cct_s_v1_global_model	S	0.5877	1701.63
cct_xs_v1_global_model	XS	0.3232	3094.21

Tabla 3.6: Comparación de desempeño base del OCR (tomado de [17]).

En este caso, se realiza optimización de hiperparámetros para determinar con qué configuración se obtienen las mejores métricas de desempeño. Los hiperparámetros incluyen el tipo e intensidad de aumento de datos, el formato de imágenes de entrada (tamaño y color), parámetros de optimizador, y parámetros de tamaño de red. Antes de iniciar el entrenamiento se seleccionan los hiperparámetros y se entrena una cantidad determinada

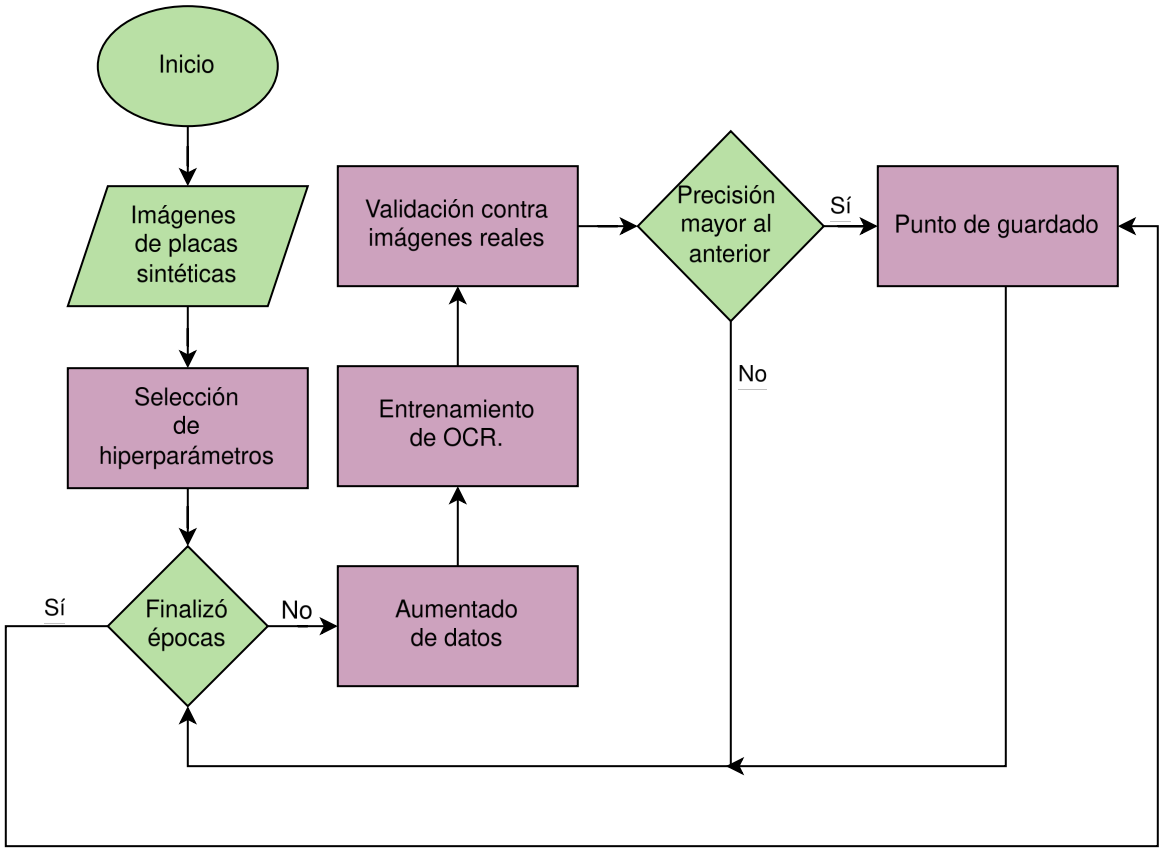


Figura 3.5: Rutina de entrenamiento del OCR.

de épocas 3.5.

A continuación se listan los principales hiperparámetros considerados en los experimentos:

1. **Optimizador**

   1.1. **Tasa de aprendizaje:** controla la magnitud del ajuste de pesos en cada iteración. Valores demasiado bajos provocan aprendizaje lento, mientras que valores muy altos pueden impedir la convergencia o causar inestabilidad numérica. *Rango:* 0.0003 – 0.001.

   1.2. **Decaimiento de pesos:** término de regularización que penaliza pesos grandes para prevenir sobreajuste y mejorar la generalización. *Rango:* 0.0002 – 0.01.

   1.3. **Tamaño de lote:** número de muestras procesadas simultáneamente en cada paso de entrenamiento. *Valores:* 16, 32, 64.

2. **Imágenes**

   2.1. **Color:** el uso de canales RGB puede aportar información cromática adicional, aunque también puede aumentar la sensibilidad a sombras o variaciones de color. *Opciones:* RGB, escala de grises.

2.2. **Resolución:** define la cantidad de información espacial disponible. Imágenes de mayor tamaño conservan más detalle pero incrementan el costo computacional. *Altura:* 32, 64, 96 píxeles. *Ancho:* 64, 128, 192 píxeles.

3. **Tamaño de red**

3.1. **Capas:** número de bloques del codificador (encoder). *Rango:* 1 – 4.

3.2. **Cabezas de atención:** número de cabezas empleadas en cada bloque de autoatención multi-cabeza. A mayor número, mayor capacidad de modelar dependencias contextuales. *Rango:* 1 – 2.

3.3. **Dimensión de proyección:** tamaño del espacio de representación interna de cada token visual. *Valores:* 32, 64, 96.

3.4. **Función de activación:** define la no linealidad en los bloques intermedios del modelo. *Opciones:* `ReLU`, `GeLU`.

3.5. **Caída en atención (Dropout de atención):** porcentaje de conexiones descartadas en los pesos de atención, empleado como regularización. *Rango:* 0.05 – 0.25.

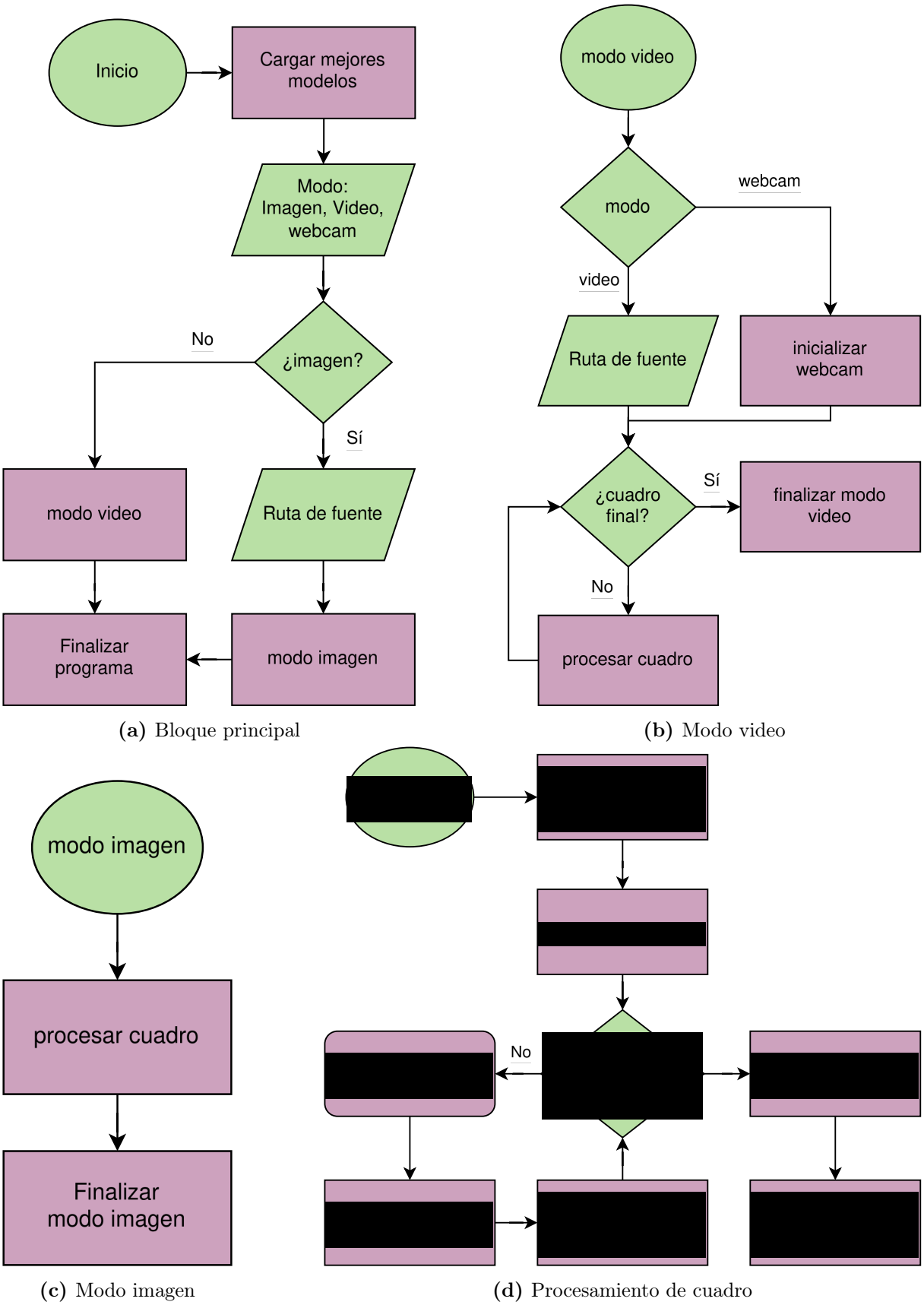
3.6. **Caída en MLP (Dropout en bloque feedforward):** porcentaje de unidades descartadas en las capas lineales posteriores al bloque de atención. *Rango:* 0.1 – 0.25.

3.4. Integración de la solución

Luego de seleccionar el mejor modelo de detección y el mejor modelo de reconocimiento de placas, se integran ambos en una única solución. Esta aplicación es capaz de procesar imágenes y video, ya sea mediante cámara en tiempo real o desde un archivo pregrabado (ver figura 3.6).

En la imagen 3.6a se muestra el bloque principal, encargado de cargar los modelos e inicializar el modo de operación correspondiente. Si se selecciona el modo imagen, se carga el archivo desde una ruta dada y se procesa un único cuadro, como se observa en la figura 3.6c. Por otro lado, si se selecciona el modo video, se procede según se ilustra en la figura 3.6b: se inicializa la cámara o se abre un archivo de video, y luego se ingresa en un bucle que continúa hasta agotar los cuadros disponibles. Dentro de este bucle se llama a la rutina encargada del procesamiento de cada cuadro.

En la rutina de procesamiento de cuadros, una vez obtenida la imagen, se extraen los puntos (X_{min}, Y_{min}) y (X_{max}, Y_{max}) que corresponden a las cajas delimitadoras. La región recortada se suministra al modelo de reconocimiento de placas, obteniéndose el texto reconocido y su nivel de confianza. Las detecciones con confianza baja (por ejemplo, menor a 0.5) se descartan. Finalmente, se dibuja la caja delimitadora y el texto asociado sobre la imagen original. Este proceso se repite hasta que se hayan procesado todos los cuadros disponibles, tal como se ilustra en la figura 3.6d.



(a) Bloque principal

(b) Modo video

(c) Modo imagen

(d) Procesamiento de cuadro

Figura 3.6: Diagrama de flujo de funcionamiento de sistema integrado

Capítulo 4

Resultados y análisis

4.1. Comparación de desempeño de detección de placas

4.1.1. Validación única (train/val split)

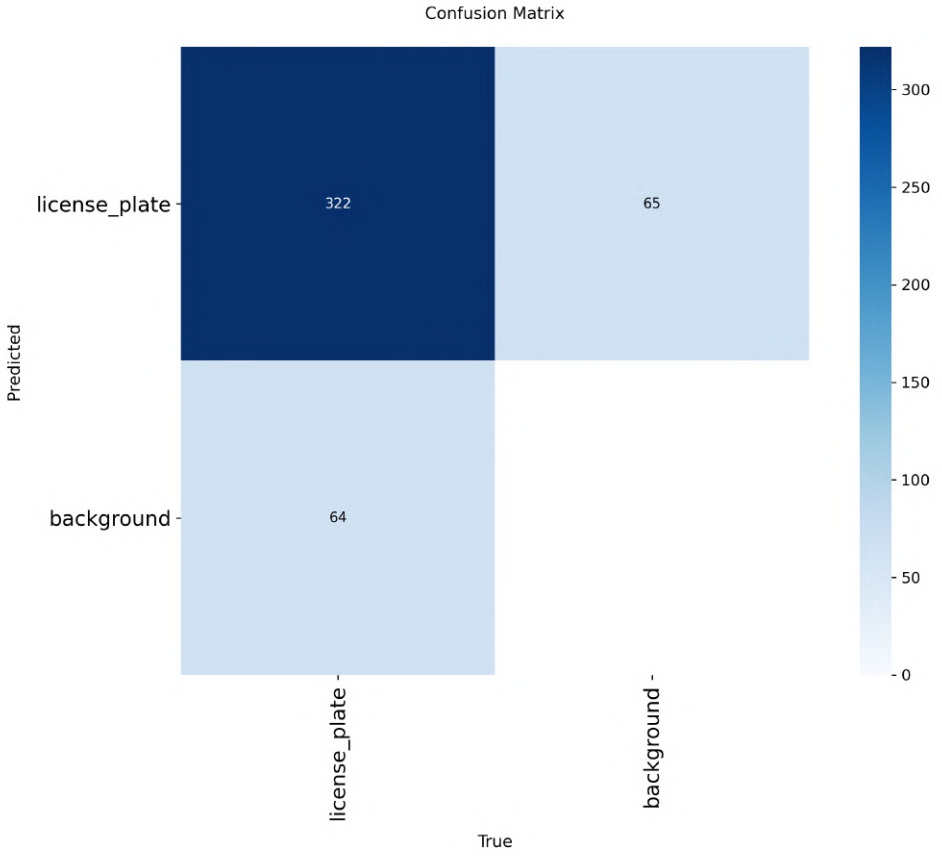
En esta sección se presentan los resultados de los experimentos realizados para la etapa de detección de placas. Se realizaron tres experimentos para comparar el desempeño del modelo de detección de placas. Los experimentos se realizaron con tres conjuntos de imágenes de entrenamiento y un único conjunto de imágenes de validación. Los conjuntos de entrenamiento fueron:

- **Conjunto 1 (CR):** 600 imágenes de vehículos costarricenses para entrenar y 300 imágenes de vehículos costarricenses para validar.
- **Conjunto 2 (CO):** 1200 imágenes de vehículos colombianos para entrenar y 300 imágenes de vehículos costarricenses para validar.
- **Conjunto 3 (Mixto):** 1200 imágenes, combinando vehículos colombianos y costarricenses, para entrenar y 300 imágenes de vehículos costarricenses para validar.

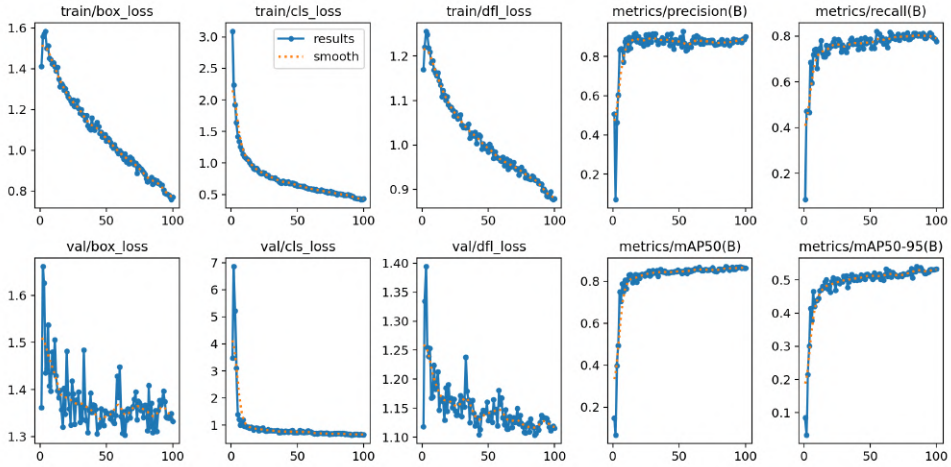
Las 300 imágenes de validación son las mismas en los tres conjuntos.

Conjunto	**Precisión**	**Exhaustividad**	**mAP@50**
Conjunto 1 (CR)	0.873	0.803	0.873
Conjunto 2 (CO)	0.870	0.653	0.774
Conjunto 3 (Mixto)	0.935	0.806	0.946

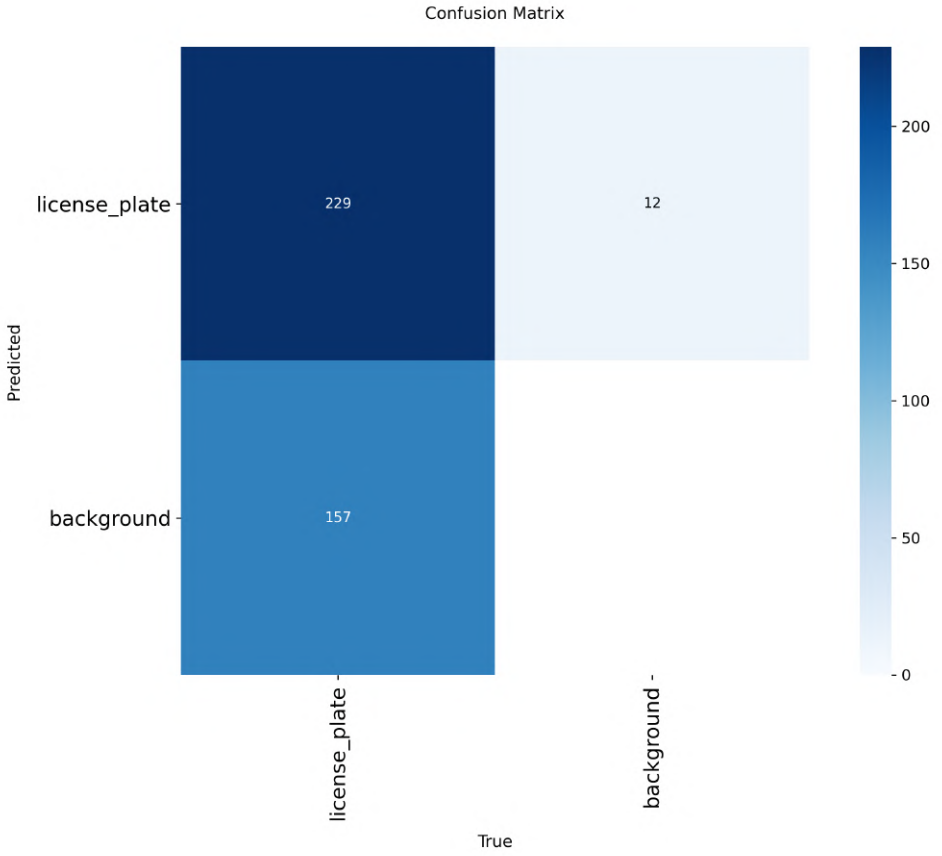
Tabla 4.1: Resultados de los experimentos con YOLOv11 para detección de placas con validación única.



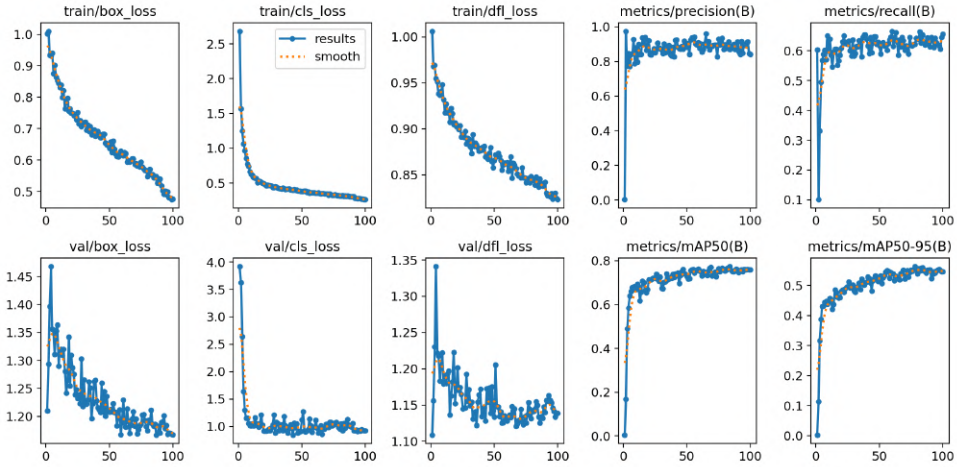
(a) Matriz de confusión del modelo experimento con conjunto 1



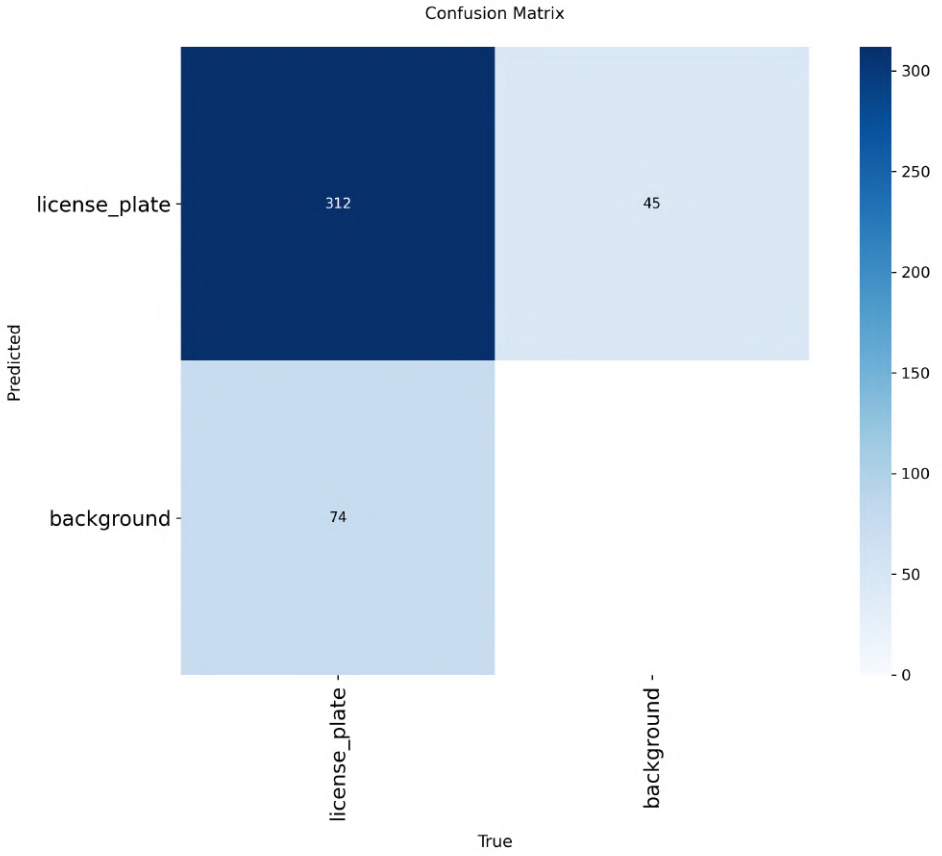
(b) Métricas del modelo experimento con conjunto 1



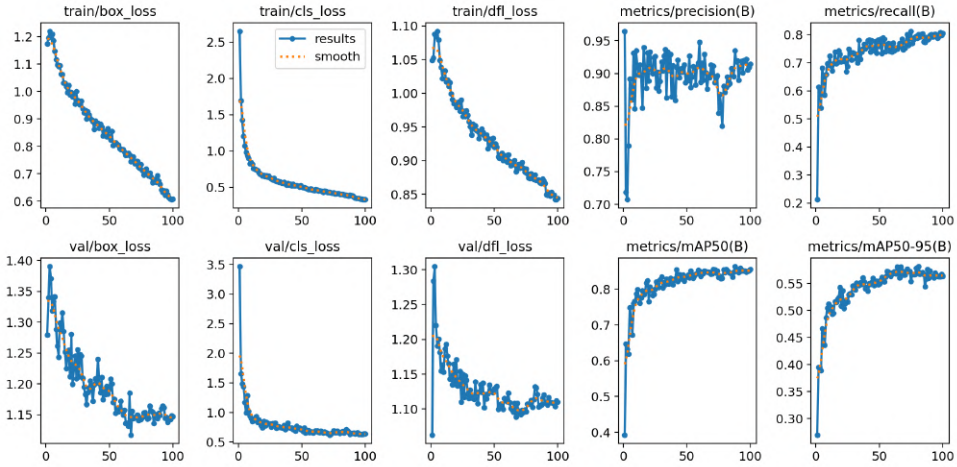
(a) Matriz de confusión del modelo experimento con conjunto 2



(b) Métricas del modelo experimento con conjunto 2



(a) Matriz de confusión del modelo experimento con conjunto 3



(b) Métricas del modelo experimento con conjunto 3

Los resultados cuantitativos de los tres experimentos se resumen en la Tabla 4.1. De acuerdo con los valores obtenidos, el modelo entrenado con el Conjunto 1 (CR) alcanzó una precisión promedio de 87.3 % y una exhaustividad (recall) de 80.3 %, con un mAP@50 de 0.870. Esto significa que el 87.3 % de las detecciones reportadas por el modelo corresponden efectivamente a placas vehiculares (precisión), mientras que el 80.3 % de todas las placas presentes en las imágenes fueron correctamente detectadas (recall). El valor de mAP@50 = 0.87 representa el promedio de precisión (AP) considerando una detección correcta cuando la superposición entre la etiqueta predicha y la etiqueta de referencia supera el 50 % (IoU $\geq$ 0.5). En la matriz de confusión (Figura 4.1a) se observa que la mayoría de las placas fueron correctamente detectadas (verdaderos positivos = 322), aunque aún se presentan 64 falsos negativos y 65 falsos positivos. Esto indica que, si bien el modelo logra detectar la mayoría de placas, todavía existen casos en los que no identifica la región correctamente.

El Conjunto 2 (CO), entrenado únicamente con imágenes de vehículos colombianos, obtuvo una precisión de 87.0 % y una exhaustividad de 65.3 %, con un mAP de 0.774. La matriz de confusión (Figura 4.2a) muestra un mayor número de errores de clasificación en comparación con el Conjunto 1, reflejando dificultades de generalización al aplicarse sobre el conjunto de prueba costarricense. En particular, se registraron 157 falsos negativos, lo cual evidencia que el modelo tiende a omitir la detección en casos donde las placas presentan características distintas a las del conjunto de entrenamiento.

Por último, el Conjunto 3 (Mixto) alcanzó los mejores resultados generales, con una precisión de 93.5 %, una exhaustividad de 80.6 % y un mAP de 0.946. En la matriz de confusión (Figura 4.3a) se observa un ligero decremento en los verdaderos positivos (312), acompañado de una reducción de falsos positivos (45), pero un ligero aumento de falsos negativos (74). Esto evidencia que la inclusión de datos de ambos dominios permitió al modelo aprender representaciones más robustas y generalizables, lo cual se tradujo en un mejor equilibrio entre precisión y exhaustividad.

La pérdida en la clasificación para el conjunto de validación (`val/cls_loss`) decrece más rápidamente para el conjunto 1 que para el conjunto 3, mientras que en el conjunto 2 la curva es más errática. Esto indica que el conjunto 2 presenta mayores dificultades para distinguir correctamente las placas, mientras que el conjunto 3 logra hacerlo, aunque requiere un mayor tiempo de entrenamiento que el conjunto 1. De manera análoga, al analizar la pérdida en la región de detección (`val/box_loss`), se observa que la curva correspondiente al conjunto 1 es la más irregular, lo que sugiere una mayor dificultad para determinar la ubicación exacta de la placa. Finalmente, la gráfica de precisión (`metrics/precision(B)`), que refleja qué tan bien clasifica el modelo una detección como placa, es más errática en el conjunto 3, a pesar de alcanzar la mejor métrica promedio. Este comportamiento se explica porque el conjunto 3 fue entrenado con un conjunto de datos más heterogéneo, lo que provoca mayor fluctuación entre lotes al ajustar los pesos para reconciliar distribuciones distintas; no obstante, mantiene un rendimiento promedio superior a los otros conjuntos (ver figuras 4.1b, 4.2b y 4.3b).

En síntesis, los resultados muestran que el entrenamiento con datos mixtos ofrece un desempeño superior respecto a los conjuntos individuales. El Conjunto 1 presenta un buen rendimiento en datos costarricenses, pero limitado en escenarios más diversos. El Conjunto 2 presenta menor capacidad de detección en el dominio local. El Conjunto 3 combina las ventajas de ambos y se perfila como la mejor configuración para integrar en el sistema propuesto.

4.1.2. Validación cruzada (k-fold)

Dado que el conjunto de validación disponible es limitado (aproximadamente 300 imágenes de placas costarricenses), una única partición de validación podría no reflejar con precisión el comportamiento real del modelo. Para obtener una evaluación más robusta y reducir el sesgo asociado a una sola división de datos, se aplicó validación cruzada con $k = 5$.

En este procedimiento, el conjunto de imágenes de placas costarricenses se dividió en cinco particiones del mismo tamaño, utilizándose cada una de ellas de forma alternada como subconjunto de validación, mientras las restantes se emplearon para el entrenamiento. De este modo, se evaluó la consistencia interna de los modelos frente a diferentes divisiones del mismo dominio, sin comparar directamente los resultados entre experimentos, ya que cada conjunto (CR, CO y Mixto) emplea datos de entrenamiento distintos.

Cabe destacar que, en el caso del conjunto mixto, las imágenes de placas internacionales (principalmente colombianas) se mantuvieron fijas durante todo el proceso, variando únicamente las particiones correspondientes a las placas costarricenses, por ser el dominio de mayor interés para la evaluación final.

Los resultados promedio y su desviación estándar se resumen en la Tabla 4.2. Dado que se entrenan cinco modelos independientes por configuración, no se presentan matrices de confusión individuales, sino únicamente las métricas agregadas.

Conjunto	**Precisión (%)**	**Exhaustividad (%)**	**mAP@0.5**
Conjunto 1 (CR)	0.920 ± 0.019	0.631 ± 0.020	0.775 ± 0.015
Conjunto 2 (CO)	0.871 ± 0.012	0.810 ± 0.014	0.872 ± 0.014
Conjunto 3 (Mixto)	0.930 ± 0.014	0.784 ± 0.017	0.863 ± 0.014

Tabla 4.2: Resultados promedio (± desviación estándar) de los experimentos con YOLO para detección de placas con validación cruzada $k = 5$.

En síntesis, los resultados muestran que el entrenamiento con datos mixtos (Conjunto 3) ofrece un desempeño superior respecto a los conjuntos individuales, tanto en validación única como en validación cruzada, confirmando que la combinación de dominios mejora la capacidad de generalización del modelo.

4.1.3. Análisis de fallos en el mejor modelo de detección de placas

Para comprender mejor la capacidad real de detección de placas del modelo es necesario analizar los casos de fallo, ya que esto permite identificar las razones por las cuales la precisión se ve afectada y plantear posibles mejoras. Este análisis es principalmente cualitativo y se realiza mediante inspección comparativa entre las predicciones del modelo y las etiquetas de referencia, tomando como base el mejor modelo (Conjunto 3). Se representa de color rojo las etiquetas de los datos, de color amarillo el reconocimiento verdadero positivo, de color azul el falso positivo, y de color verde el falso negativo.

La mayoría de los casos de error corresponden a falsos positivos por ausencia de anotación: el modelo detecta una placa, pero en las etiquetas de referencia no existe registro de ella. En realidad, estos casos reflejan deficiencias en la anotación de los datos, dado que efectivamente hay una placa presente en la imagen. Por lo tanto, la predicción es correcta, pero se contabiliza como error debido a la falta de etiqueta, lo que afecta artificialmente la métrica de precisión. Ejemplos de este tipo de fallos se muestran en la Figura 4.4.

En menor medida, se identificaron casos de falsos negativos reales, donde el modelo no logra detectar una placa presente en la imagen. Estos errores corresponden a situaciones adversas, como placas parcialmente ocluidas, muy pequeñas debido a la distancia de captura o con iluminación adversa. Ejemplos de estos casos se muestran en la Figura 4.5.

Finalmente, se encontraron muy pocos casos de falsos positivos genuinos, en los que el modelo predice una placa en una región donde no existe ninguna. Estos errores suelen deberse a patrones visuales similares a los de una placa, como bordes de objetos o elementos rectangulares en los vehículos. Un ejemplo de este tipo de fallo se presenta en la Figura 4.6.

4.1.4. Evaluación respecto a modelo base

Con el fin de validar la mejora del sistema propuesto, se compararon los resultados del mejor modelo YOLO (entrenado con el Conjunto 3, mixto) frente a un modelo base tomado del proyecto anterior [2], y YOLO "puro", sin ajuste. La tabla 4.3 resume las métricas obtenidas.

Modelo	**Precisión**	**Exhaustividad**	**mAP**
YOLO sin ajuste	0	0	0
Modelo base	0.811	0.703	0.814
YOLO propuesto (Conjunto 3)	0.935	0.806	0.946

Tabla 4.3: Comparación de desempeño entre el modelo base y el modelo YOLO propuesto.

Los resultados muestran que el modelo YOLO propuesto logra una mejora significativa



(a) Detección de placa falsa positiva en segundo auto blanco



(b) Detección de placa falsa positiva en auto negro al fondo



(c) Detección de placa falsa positiva en auto trasero de la derecha



(d) Detección de placa falsa positiva en auto de la izquierda

Figura 4.4: Ejemplos de falsos positivos de detección de placas



(a) No detección de placa en auto trasero de la izquierda



(b) No detección de placa en auto lejano

Figura 4.5: Ejemplos de falsos negativos de detección de placas



(a) Confusión de QR con un placa

Figura 4.6: Ejemplo de falsos positivos completo

respecto al modelo base, y que el modelo YOLO original no es apto para detectar placas sin ajustes previos. En cuanto a la detección de placas, el mAP@0.5 aumentó en 0.132 (de 0.814 a 0.946), superando ampliamente el umbral de 0.03 establecido como referencia. Asimismo, la precisión y la exhaustividad se incrementaron en 0.124 y 0.103 respectivamente, reflejando una mayor capacidad del modelo para localizar y clasificar correctamente las placas en imágenes de distintos dominios.

En conjunto, estos resultados confirman que el modelo YOLO entrenado con datos mixtos no solo mejora la detección de las placas, sino que también proporciona un reconocimiento más robusto frente a variaciones de iluminación, ángulo y fondo. Esto valida la hipótesis de que la combinación de datos de diferentes fuentes aporta una mejor generalización y constituye un avance significativo respecto a la solución inicial. Finalmente, a modo de ejemplo, la figura 4.7 muestra algunas imágenes de detección que ilustran el funcionamiento del sistema en esta etapa.



Figura 4.7: Ejemplo de detección de placas en imágenes

4.2. Características de las placas sintéticas generadas

Con el fin de suplir la falta de datos reales, se generan imágenes sintéticas de placas vehiculares. Estas imágenes son usadas como base para el entrenamiento del modelo de reconocimiento de placas (OCR), mientras que la validación posterior se realiza con imágenes reales. A continuación, se muestran algunos ejemplos de placas generadas en figura 4.8



Figura 4.8: Ejemplos de placas sintéticas



(a) **(b)** **(c)**

(d) **(e)** **(f)**

Figura 4.9: Ejemplos de aumentado de datos

Para ilustrar el funcionamiento del sistema de aumento de datos, se presentan ejemplos en la figura 4.9. En la imagen 4.9a se observa una distorsión física en los bordes de la placa y sus caracteres, mientras que la imagen 4.9b muestra un incremento en el brillo. La imagen 4.9c exhibe alta difuminación, y la imagen 4.9d presenta artefactos de color que dificultan parcialmente la visualización. El ejemplo 4.9e combina las transformaciones anteriores con mayor intensidad, y la imagen 4.9f evidencia el efecto de la transformación

en perspectiva. Todas las imágenes incluyen además una rotación, la transformación más común aplicada.

4.3. Comparación de desempeño en reconocimiento de placas

Para la etapa de reconocimiento de caracteres (OCR) se realizaron en primer lugar experimentos exploratorios con distintos hiperparámetros relacionados con la entrada de la imagen, la arquitectura del modelo y el optimizador de entrenamiento. El objetivo de esta fase inicial fue identificar un conjunto razonable de parámetros base antes de afinar el sistema con el aumento de datos.

4.3.1. Primera ronda de experimentos: exploración de hiperparámetros

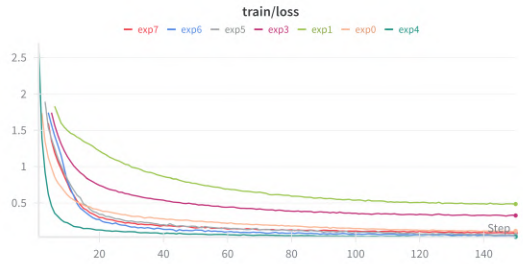
En total se llevaron a cabo 8 experimentos (incluyendo la línea base), seleccionando combinaciones aleatorias dentro de los rangos de la Tabla 4.4. Además, usando el programa de generación de datos sintéticos, se generaron 8500 imágenes aleatorias, dejando 730 imágenes de placas reales para validación.

Parámetro	**Rango de prueba**
Capas	1 – 4
Cabezas	1 – 2
Dimensión de proyección	32, 64, 96
Función de activación	gelu, relu
Caída de atención	0.05 – 0.25
Caída de MLP	0.1 – 0.25
Alto de imagen	32, 64, 96
Ancho de imagen	64, 128, 192
Color de imagen	RGB, escala de grises
Tamaño de lote	16, 32, 64
Decaimiento de pesos	0.0002 – 0.01
Tasa de aprendizaje	0.0003 – 0.001

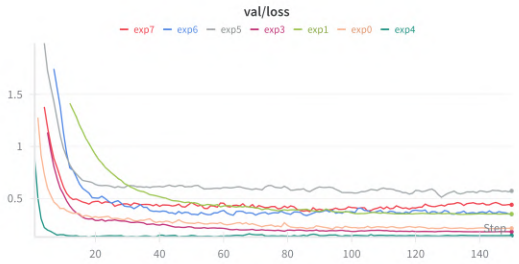
Tabla 4.4: Rangos de hiperparámetros explorados en la primera ronda de experimentos.

En la figura 4.10 se muestran las curvas de pérdida y de precisión, tanto en entrenamiento como en validación. Estas evidencian una alta variabilidad entre configuraciones, lo cual confirma la necesidad de esta fase exploratoria.

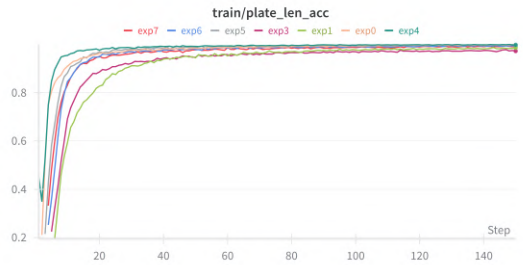
Luego, en la figura 4.11 se muestran los resultados de precisión de reconocimiento de



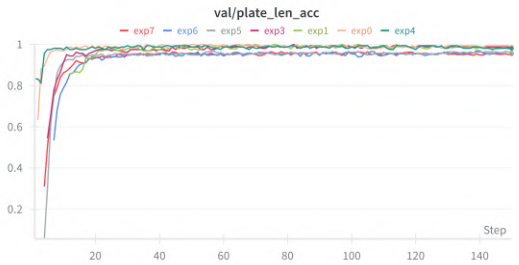
(a) Función de pérdida en conjunto de entrenamiento



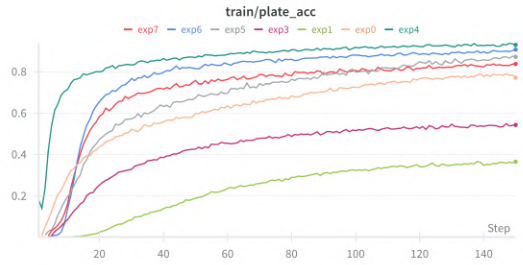
(b) Función de pérdida en conjunto de validación



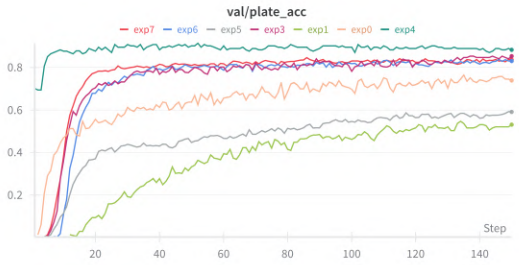
(c) Precisión de reconocimiento de longitud de placa en conjunto de entrenamiento



(d) Precisión de reconocimiento de longitud de placa en conjunto de validación



(e) Precisión de reconocimiento de placa en conjunto de entrenamiento



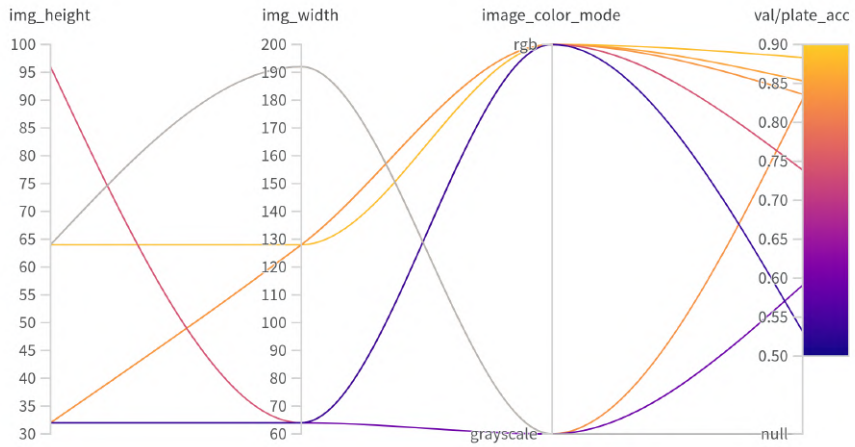
(f) Precisión de reconocimiento de placa en conjunto de validación

Figura 4.10: Métricas de aprendizaje en primer conjunto de experimentos

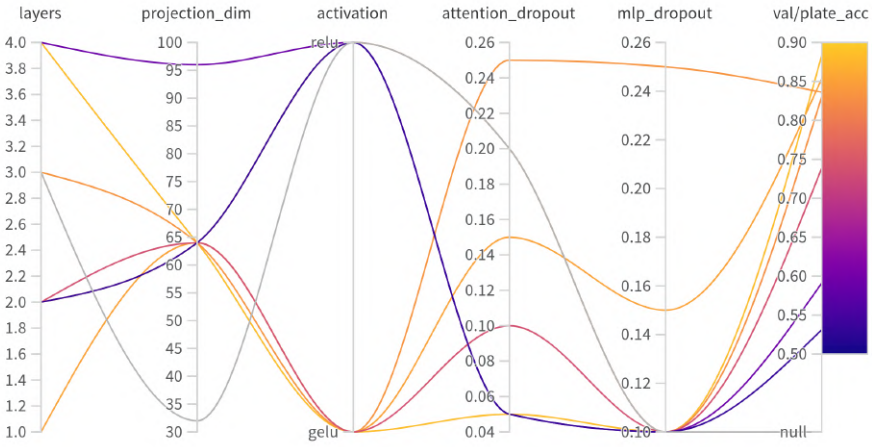
placa en validación contra los hiperparámetros. De estas gráficas se concluye que el mejor conjunto de hiperparámetros es el mostrado en la tabla 4.5, por lo que estos se mantienen fijos para las siguientes pruebas.

En esta primera ronda se observó un desempeño alto en las métricas de entrenamiento, con varios modelos alcanzando precisiones de categoría y placa cercanas o iguales a 1.0, indicando un ajuste completo al conjunto de entrenamiento. Sin embargo, las métricas de validación revelan diferencias significativas en la capacidad de generalización.

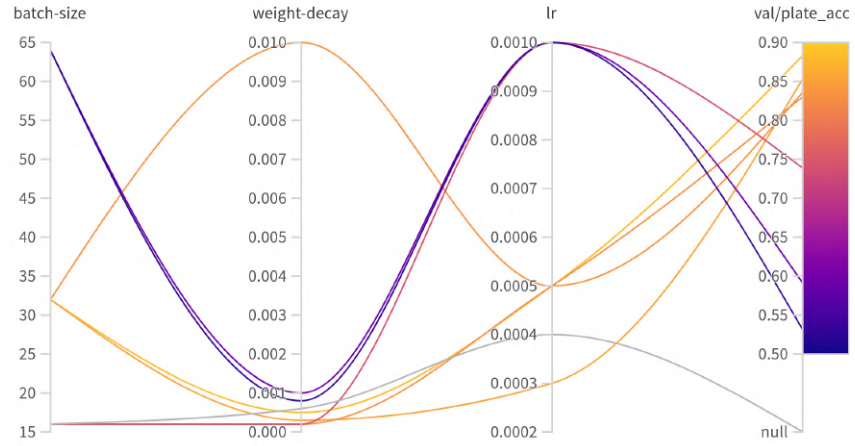
Los experimentos 4, 3 y 7 presentaron las mejores métricas de validación globales, alcanzando precisiones de placa superiores a 0.84 y pérdidas relativamente bajas. Por ejemplo, el experimento 4 logró una precisión de validación de 0.9144. En contraste, el experimento 5 mostró una caída notable en la precisión de placa (0.5926)y un aumento de la pérdida, indicando sobreajuste o una combinación inadecuada de hiperparámetros, como mayor número de cabezas de atención y proyección.



(a) Hiperparámetros de la imagen contra precisión de reconocimiento de placa



(b) Hiperparámetros del modelo contra precisión de reconocimiento de placa



(c) Hiperparámetros del entrenamiento contra precisión de reconocimiento de placa

Figura 4.11: Comparación de hiperparámetros contra precisión en validación

El experimento 7, aunque obtuvo buenos resultados en entrenamiento, presentó una ligera disminución en validación respecto al experimento 3, probablemente por un exceso de caída (0.25). Por otro lado, el experimento 6 mantuvo un equilibrio razonable entre entrenamiento y validación, mostrando pérdida moderada y buena estabilidad, lo que indica una configuración más robusta.

En general, se observó que configuraciones con 2–3 capas y 1 cabeza de atención ofrecieron mejor desempeño general. El uso de activación GELU y valores moderados de caída (0.1–0.15) favoreció la estabilidad y precisión. Los modelos con tamaño de imagen 32×64 en color RGB y batch de 32–64 mostraron mejor capacidad de generalización.

Así, el experimento 4 se destaca como el más equilibrado entre rendimiento y estabilidad, constituyendo un buen punto de partida para ajustes finos en la siguiente ronda de optimización. Los parámetros fijos seleccionados se resumen en la tabla 4.5.

Parámetro	**Valor**
Capas	4
Cabezas de atención	1
Dimensión de proyección	64
Función de activación	GELU
Caída de atención	0.05
Caída de MLP	0.1
Altura de imagen	64
Ancho de imagen	128
Modo de color	RGB
Tamaño de lote	32
Decaimiento de pesos	0.0005
Tasa de aprendizaje	0.0005

Tabla 4.5: Mejor conjunto de hiperparámetros de entrada, optimizador y modelo.

Parámetro	**Valor**
Sistema	Cluster Kabré (TEC)
Nodo	Nukwa
Sistema operativo	AlmaLinux 9
GPU	NVIDIA Tesla V100 / L40S
Procesador	20–24 núcleos (2 hilos por núcleo)
Memoria RAM	32–256 GB
Gestor de tareas	SLURM
Tipo de ejecución	Asignación automática de nodo único

Tabla 4.6: Configuración del sistema utilizado para la ejecución de los experimentos de entrenamiento.

Los experimentos de entrenamiento y validación del modelo OCR se ejecutaron en el cluster Kabré del Centro Nacional de Alta Tecnología (CeNAT), específicamente en los nodos de aprendizaje automático Nukwa. Cada nodo dispone de una GPU NVIDIA Tesla V100 o NVIDIA L40S, procesadores multinúcleo y gran capacidad de memoria, según lo indicado en la tabla 4.6. Esta infraestructura permitió realizar los experimentos de manera reproducible y con tiempos de entrenamiento razonables para modelos de visión por computadora, según lo mostrado en tabla 4.7.

Métrica	Tiempo
Tiempo medio de entrenamiento	11h 23m 22s
Variabilidad del tiempo de entrenamiento	26m 47s

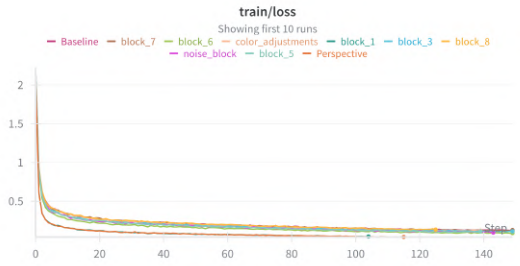
Tabla 4.7: Resultados de duración de los experimentos de entrenamiento

4.3.2. Segunda ronda de experimentos: optimización de aumentos de datos

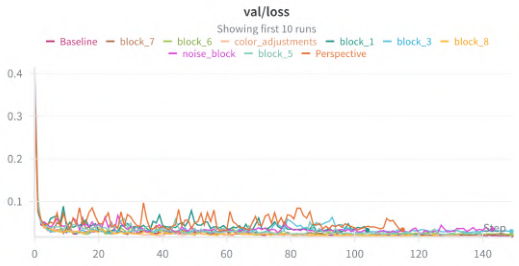
Con los parámetros principales ya fijados, la segunda ronda de experimentos se enfocó en evaluar el impacto de los esquemas de aumento de datos, modificando tanto el tipo como la intensidad de los mismos. En esta fase se probaron técnicas como rotaciones, cambios de brillo y contraste, ruido aditivo y distorsiones geométricas, entre otras. Las gráficas resultantes de los experimentos se muestran en la figura 4.12.

Los experimentos realizados corresponden a un análisis ablativo sobre la base de aumentación definida para el OCR de matrículas. Cada variante elimina total o parcialmente un conjunto de transformaciones, con el fin de cuantificar su aporte individual al rendimiento del modelo. Los resultados se resumen en la tabla 4.8. A continuación, se describe el significado de cada configuración:

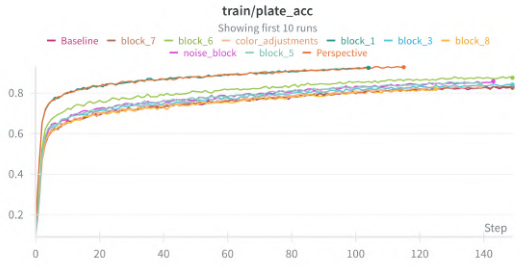
- **block_1:** eliminación del bloque de transformaciones geométricas iniciales (*Affine, Perspective*).
- **block_2:** remueve distorsiones no lineales (*ElasticTransform, GridDistortion, OpticalDistortion*).
- **block_3:** suprime el ajuste de brillo y contraste (*RandomBrightnessContrast*).
- **block_4:** elimina el bloque de desenfoques y perturbaciones de canal (*MotionBlur, GaussianBlur, ChannelShuffle*).
- **block_5:** remueve el bloque de ruido y degradación (*GaussNoise, MultiplicativeNoise, ImageCompression, PixelDropout*).



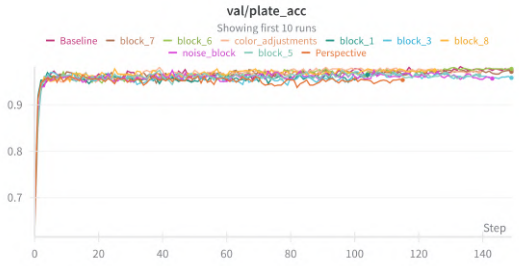
(a) Pérdida en entrenamiento



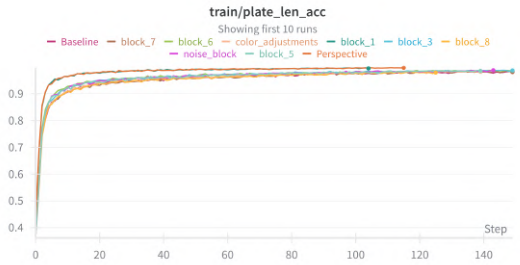
(b) Pérdida en validación



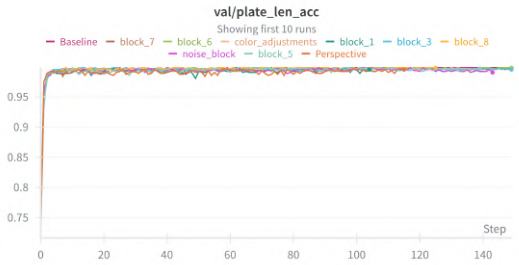
(c) Precisión de reconocimiento de placa completa en entrenamiento



(d) Precisión de reconocimiento de placa completa en validación



(e) Precisión de reconocimiento de cantidad de caracteres en entrenamiento



(f) Precisión de reconocimiento de cantidad de caracteres en validación

Figura 4.12: Métricas de entrenamiento y validación de aumentado de placas con aumentado de datos

- **block_6:** suprime las pérdidas de información locales y de resolución (*CoarseDropout, Downscale*).
- **block_7:** elimina los ajustes de color (*HueSaturationValue, RGBShift*).
- **block_8:** quita la corrección de gamma (*RandomGamma*).
- **block_9:** elimina la conversión ocasional a escala de grises (*ToGray*).
- **Affine:** elimina la transformación *Affine*, conservando sólo *Perspective*.
- **Perspective:** elimina la transformación *Perspective*, conservando sólo *Affine*.
- **blur_effects:** remueve completamente los efectos de desenfoque.
- **noise_block:** suprime todos los ruidos y degradaciones de compresión o pérdida local.

- **color_adjustments:** elimina las transformaciones de color y gamma.

El siguiente análisis resume los resultados experimentales y las implicaciones de cada eliminación: El modelo base emplea todas transformaciones y alcanza una precisión de validación de 0.9844, sirviendo como referencia para el resto de variantes. Cuando se elimina el bloque 6 — encargado de la reducción de resolución y pérdidas locales de información (*Downscale, CoarseDropout*)— la precisión cae ligeramente (–0.0036), lo que evidencia que dichas perturbaciones favorecen la generalización ante imágenes de baja calidad. De forma similar, al suprimir los ajustes de color y corrección de gamma (color_adjustments) se observa una pérdida equivalente (–0.0036), indicando que las variaciones cromáticas contribuyen a la estabilidad del modelo bajo distintos entornos lumínicos. La ausencia de desenfoques (blur_effects) provoca un descenso más marcado (–0.0052), mostrando que la simulación de movimiento o desenfoque de foco es relevante para el reconocimiento robusto de caracteres. Por su parte, eliminar el bloque 8, responsable de la corrección de gamma, también reduce el desempeño (–0.0068), lo cual confirma la importancia del control de luminancia.

Cuando se eliminan las distorsiones no lineales (block_2, que agrupa *ElasticTransform, GridDistortion* y *OpticalDistortion*), la precisión disminuye en igual magnitud (–0.0068), reflejando que dichas deformaciones ayudan al modelo a adaptarse a curvaturas y aberraciones ópticas comunes. Una tendencia similar se observa al remover la transformación *Affine*, manteniendo sólo *Perspective*, donde el descenso de rendimiento también es de –0.0048, confirmando la relevancia de las variaciones geométricas básicas. La eliminación

Experimento	**Precisión Val.**	**Precisión Train.**	**Δ vs Baseline**
Baseline	0.9844	0.8339	0.0000
block_6	0.9808	0.8798	-0.0036
color_adjustments	0.9808	0.8326	-0.0036
blur_effects	0.9792	0.8596	-0.0052
block_8	0.9776	0.8205	-0.0068
block_2	0.9776	0.8412	-0.0068
Affine	0.9776	0.8231	-0.0068
block_7	0.9760	0.8319	-0.0084
block_4	0.9760	0.8576	-0.0084
block_5	0.9744	0.8562	-0.0100
block_9	0.9744	0.8178	-0.0100
noise_block	0.9744	0.8582	-0.0100
block_3	0.9712	0.8440	-0.0132
block_1	0.9696	0.9271	-0.0148
Perspective	0.9647	0.9302	-0.0196

Tabla 4.8: Comparación de precisión entre el modelo *Baseline* y las variantes con diferentes transformaciones de datos.

del bloque 7, que abarca los ajustes de tono, saturación y canales de color, ocasiona una caída más notable (–0.0084), lo que refuerza el papel del color en la generalización. Un efecto comparable se presenta al suprimir el bloque 4, que incluye desenfoques y mezcla de canales, con igual pérdida de precisión (–0.0084). Por otro lado, quitar el bloque 5 —responsable de la generación de ruido, compresión y *PixelDropout*— produce una disminución de –0.0100, lo cual subraya el valor de las degradaciones sintéticas para mejorar la tolerancia del OCR ante artefactos reales. El mismo impacto se observa al eliminar la conversión ocasional a escala de grises (block_9) o al remover todo el bloque de ruido (noise_block), confirmando su función como regularizador. El ajuste de brillo y contraste (block_3) resulta ser uno de los componentes más críticos: su ausencia reduce la precisión en –0.0132, evidenciando su papel clave para la tolerancia frente a condiciones lumínicas variables. De igual manera, la eliminación de las transformaciones geométricas iniciales (block_1), que incluyen *Affine* y *Perspective*, produce una caída aún mayor (–0.0148), señalando que las modificaciones espaciales son esenciales para el aprendizaje robusto. Finalmente, suprimir únicamente la transformación *Perspective* ocasiona la mayor degradación de desempeño (–0.0196), lo que demuestra que la inclinación del plano de la matrícula es uno de los factores de mayor impacto en la capacidad de generalización del modelo.

En conjunto, los resultados del análisis ablativo confirman que las transformaciones geométricas —especialmente *Perspective* y *Affine*— junto con los ajustes de brillo y contraste son los componentes más determinantes en la robustez del OCR. En cambio, la supresión de perturbaciones menores de color o ruido produce efectos moderados. El modelo completo logra el mejor equilibrio entre generalización y precisión (0.9844 en validación), mientras que las variantes simplificadas tienden a perder tolerancia frente a degradaciones reales de imagen. Además, esto indica que el aumento de datos no solo mejora la precisión en validación, pasando de 0.914 a 0.984, sino que además la combinación de los diferentes aumentos de datos es la que logra mejorar la capacidad de reconocimiento del sistema. En adición, ninguno de los aumentos utilizados llevó a degradaciones del desempeño.

4.3.3. Análisis de fallos del OCR

Utilizando el mejor de los modelos, se realiza el análisis de los casos de fallo, para determinar, cualitativamente, las características de los fallos existentes; esto permite planificar mejoras posibles en futuros experimentos y las limitaciones del sistema. Algunos de los casos de fallo se muestran en las figuras 4.13.

En las imágenes 4.13c, 4.13e y 4.13f el motivo principal de fallo es atribuible al efecto de reflexión secundaria (también conocido como ghosting), caracterizado por la aparición de una réplica tenue y ligeramente desplazada de la placa original. Esta superposición introduce duplicidad de bordes y solapamiento de contornos de caracteres, lo que confunde al OCR.

Los casos 4.13a, 4.13b y 4.13g son atribuibles a problemas de perspectiva. En 4.13a la



(a) Fallo en primera K **(b)** Fallo en el 8 **(c)** Fallo en el 1

(d) Fallo en el segundo 7 **(e)** Fallo en ambos 5 **(f)** Fallo completo

(g) Fallo en primer 6 y el 3 **(h)** Fallo completo **(i)** Fallo en la X

(j) Fallo completo **(k)** Fallo completo

Figura 4.13: Casos de fallo en el reconocimiento de placas

cercanía entre el borde de la placa y el trazo de la letra "K", ambos de color similar, provoca ambigüedad en la agrupación de contornos. En 4.13b la vista elevada acorta el semicírculo superior del "8", produciendo que el carácter se interprete como un "6". En 4.13g el ángulo de inclinación es particularmente pronunciado y lo cual puede afectar la forma en que el OCR aprendió los rasgos característicos de los dígitos.

El fallo mostrado en 4.13i parece originarse en la baja calidad de la imagen —alto grado de desenfoque y pérdida de detalle— que provoca la confusión entre rasgos similares (por ejemplo, "X" ↔ "K") al perderse los bordes finos de los caracteres. Los casos 4.13h y 4.13k están relacionados con condiciones de iluminación: una iluminación excesiva reducen el contraste en los bordes de los caracteres y provocan el fallo completo del reconocimiento de la placa.

Finalmente, el caso 4.13j es interesante porque, aunque la placa presenta ligeras abolladuras que no debería impedir la lectura, se producen múltiples errores en distintos dígitos. Esto sugiere la presencia de factores combinados y requiere un análisis posterior más exhaustivo.

4.3.4. Comparación respecto a modelo base

De igual forma que con el detector de placas, con el fin de validar el desempeño del sistema propuesto y su mejora, se compara este con los resultados ofrecidos frente a algún modelo base definido. Se utiliza para esto el modelo EasyOCR usado en el proyecto anterior [2], y Fast-Plate-OCR sin ajuste fino. La tabla 4.9 resume las métricas obtenidas.

Modelo	Precisión de placa	Precisión de longitud
EasyOCR	0.037	0.037
Fast-Plate-OCR sin ajuste	0.755	0.807
OCR propuesto	0.984	0.997

Tabla 4.9: Comparación de desempeño entre el modelo base y el modelo OCR propuesto.

Los resultados muestran una mejora significativa del sistema propuesto respecto a los modelos base. Mientras que EasyOCR alcanza apenas un 3.7 % de precisión de placa y precisión de longitud, el modelo Fast Plate OCR sin ajuste fino logra un desempeño moderado con 75.5 % y 80.7 %, respectivamente. En contraste, el modelo propuesto, tras el proceso de ajuste fino y optimización de aumento de datos, alcanza una precisión de placa del 98.4 % y una precisión de longitud del 99.7 %. Estas cifras superan ampliamente los criterios establecidos de mejora ($\geq$ 10 % para precisión completa y $\geq$ 5 % para reconocimiento parcial), validando que el sistema no solo incrementa la exactitud en el reconocimiento completo de placas, sino también su robustez frente a variaciones en los datos de entrada. En conjunto, los resultados confirman que el ajuste fino del modelo Fast Plate OCR permite obtener un OCR más preciso, estable y generalizable que las alternativas empleadas previamente.

Finalmente, a modo de ejemplo, en las figuras en 4.14 se muestran ejemplos reconocimiento de placas que ilustran el funcionamiento y desempeño de esta etapa del sistema.



Figura 4.14: Ejemplo de reconocimiento de placas.

4.4. Evaluación del desempeño del sistema integrado

Para validar el rendimiento operativo del sistema completo —es decir, la integración del detector YOLO y el reconocedor Fast-Plate-OCR—, se realizaron mediciones de latencia y velocidad de procesamiento en la aplicación final. El sistema se ejecutó en modo video, procesando un flujo continuo de cuadros con resolución 1080×720 píxeles. La medición de tiempos se realizó mediante el uso de la función `time.time()` de Python, que permitió calcular la latencia por cuadro y la velocidad promedio en cuadros por segundo (FPS).

Las pruebas se realizaron en un entorno sin aceleración por GPU, renderizando el video en la interfaz gráfica, utilizando los modelos originales en formato `.pt`, y sin optimización ni conversión a formatos `ONNX`. El sistema operativo y hardware de prueba se detallan en la tabla 4.10.

Parámetro	**Valor**
Sistema operativo	Arch Linux x86_64
Procesador	Intel® Core™ i5-7200U (4 núcleos, 3.10 GHz)
GPU	Intel HD Graphics 620 (integrada, 1.0 GHz)
Memoria RAM	8 GB DDR4
Disco	HDD 930 GB (btrfs)

Tabla 4.10: Configuración del sistema utilizado para las pruebas de desempeño.

Durante la ejecución, se obtuvieron los resultados resumidos en la tabla 4.11.

Métrica	**Valor medido**
Latencia promedio por cuadro	$299{,}6 \pm 69{,}7$ ms
Latencia mínima	174.11 ms
Latencia máxima	430.68 ms
Velocidad promedio	$3{,}84 \pm 0{,}79$ FPS
Cuadros procesados	4758
Cuadros con detección	2047 (43.0 %)
Videos analizados	13

Tabla 4.11: Resultados de desempeño del sistema integrado (YOLO + Fast-Plate-OCR).

Los resultados reflejan un rendimiento limitado por la ausencia de aceleración por GPU y el uso de modelos en formato sin optimizar, y hardware de baja gama. El sistema logró mantener una velocidad promedio de 3.84 FPS y una latencia de 300 ms por cuadro. Se espera que el desempeño mejore significativamente mediante el uso de versiones optimizadas de los modelos en formato `ONNX` y ejecución en hardware con aceleración CUDA.

Finalmente, igual que en los casos anteriores, se muestran en la figura 4.15 ejemplos del sistema completo en funcionamiento.



Figura 4.15: Ejemplo de funcionamiento del sistema

Capítulo 5

Conclusiones

Se cumplió el objetivo del proyecto, que consistía en diseñar un prototipo de sistema automático para la detección y reconocimiento de caracteres en placas vehiculares costarricenses mediante técnicas de visión por computadora y aprendizaje profundo. Además, se logró establecer una arquitectura basada de dos etapas basadas en aprendizaje profundo —YOLOv11 para la detección de placas vehiculares y Fast-Plate-OCR para el reconocimiento óptico de caracteres— evitando así las limitaciones de los sistemas tradicionales basados en segmentación de caracteres. La integración en un único flujo de procesamiento permitió validar el correcto funcionamiento del sistema completo, desde la detección de las placas hasta el reconocimiento de texto correspondiente.

En cuanto a la preparación del conjunto de datos, se generaron más de 8000 imágenes sintéticas, junto con un esquema de aumento de datos con los cuales entrenar el módulo de reconocimiento de caracteres. Esta estrategia permitió ampliar la variabilidad de entrenamiento sin depender de exclusivamente de la recolección de imágenes reales.

Los mejores resultados experimentales en la detección de placas se consiguieron al enriquecer el conjunto de datos de entrenamiento con placas vehiculares de otro país, esto le permite al modelo generalizar mejor el concepto de "placa" y evita el sobreajuste a las imágenes de vehículos costarricenses. De manera análoga, el mejor resultado experimental en el reconocimiento de placas vehiculares se obtuvo al generar aumentos de datos con distintos tipos de aumento, al eliminar tipos de aumento individualmente, la capacidad de generalización del sistema decae.

Respecto a la evaluación del sistema, se establecieron métricas de desempeño y se comparó el sistema propuesto con modelos base. Los resultados experimentales demuestran un desempeño sobresaliente en ambos módulos principales. El detector YOLO alcanzó una precisión promedio del 93.5 %, y un mAP@0.5 de 0.946, superando holgadamente el criterio de mejora mínima del 3 %, respecto al modelo base. Por su parte, el reconocedor de placas logró una precisión del 98.4 %, superando en más de 90 puntos porcentuales al modelo base (3.7 %). Estos resultados evidencian la efectividad del entrenamiento con datos aumentados y el diseño modular del sistema.

En cuanto al desempeño del sistema integrado, las pruebas sobre secuencias de video mostraron una latencia promedio de 300 ms por cuadro y una velocidad de 3.84 FPS en una máquina sin aceleración por GPU. Si bien estas métricas no corresponden a operación en tiempo real de alta velocidad, resultan adecuadas para el contexto del estacionamiento del Campus, donde los vehículos se desplazan a baja velocidad y permanecen dentro del campo visual el tiempo suficiente para ser procesados. Se espera que el uso de modelos optimizados (`ONNX`) y hardware con aceleración CUDA incremente significativamente la velocidad del sistema, manteniendo su precisión.

En términos generales, el desarrollo permitió cumplir los objetivos planteados: diseñar, entrenar e integrar un sistema robusto de detección y reconocimiento de placas vehiculares, superando ampliamente las métricas base en precisión y exactitud completa. El acceso al código fuente de este proyecto se encuentra en el siguiente *enlace* a un repositorio de control de versiones.

5.1. Recomendaciones

Este sistema presenta limitaciones que, si se desea utilizar en la implementación posterior del sistema gestión vehicular, es recomendable abordar. Con la escasa cantidad de datos disponibles, la validación del modelo de detección de placas puede estar sesgada, haciendo necesario incrementar la cantidad de datos disponibles; y en la misma línea, en la sección de evaluación de fallos se encontró que el desempeño de los sistemas de detección y reconocimiento de placas se encuentra afectado debido a errores en las etiquetas de los datos, por lo que conviene revisar las etiquetas del conjunto de datos empleado. Por otro lado, en lo que respecta al reconocimiento de placas, en el conjunto de datos no se encuentran todos los tipos de placas vehiculares de Costa Rica. Esto impone limitantes a los escenarios donde el sistema puede ser utilizado. Es necesario, por lo tanto, no solo enriquecer el conjunto de datos con automóviles que tengan más tipos de placas diferentes, sino también hacer más robusto el programa de generación sintética de placas de entrenamiento, para que sea capaz de generar esas placas. En adición, conviene realizar una mayor exploración en el aumento de datos, de manera que se puedan tratar los casos de fallos encontrados en los experimentos. Finalmente, en la integración del sistema completo, resulta conveniente evaluar el rendimiento con el sistema optimizado y en el hardware final donde se ejecutará la aplicación.

En síntesis, el sistema propuesto constituye una base sólida para aplicaciones prácticas de reconocimiento automático de placas vehiculares, demostrando mejoras sustanciales respecto a modelos previos y abriendo la posibilidad de futuras extensiones hacia escenarios en tiempo real o sistemas inteligentes de gestión vehicular.

Bibliografía

[1] F. M. Bolaños, *Campus TEC San José estrena instalaciones*, Acceso el 24 de junio de 2025, 2021. dirección: https://www.tec.ac.cr/hoyeneltec/2021/06/16/campus-tec-san-jose-estrena-instalaciones.

[2] B. Cordero y R. Martinez, «Desarrollo de un sistema de reconocimiento automático de placas vehiculares para la gestión inteligente del parqueo en el Campus Tecnológico Local de San José.,» Informe de curso Taller Integrador.

[3] S.-R. Wang, H.-Y. Shih, Z.-Y. Shen y W.-K. Tai, *End-to-End High Accuracy License Plate Recognition Based on Depthwise Separable Convolution Networks*, 2022. arXiv: 2202.10277 [cs.CV]. dirección: https://arxiv.org/abs/2202.10277.

[4] C.-C. Peng, C.-J. Tsai, T.-Y. Chang, J.-Y. Yeh, H. Dai y M.-H. Tsai, «A Fast and Noise Tolerable Binarization Method for Automatic License Plate Recognition in the Open Environment in Taiwan,» *Symmetry*, vol. 12, n.º 8, 2020, ISSN: 2073-8994. DOI: 10.3390/sym12081374. dirección: https://www.mdpi.com/2073-8994/12/8/1374.

[5] V. Kamat y S. Ganesan, «An efficient implementation of the Hough transform for detecting vehicle license plates using DSP'S,» en *Proceedings Real-Time Technology and Applications Symposium*, 1995, págs. 58-59. DOI: 10.1109/RTTAS.1995.516201.

[6] M. Y. Arafat, A. S. Mohd Khairuddin y R. Paramesran, «Connected component analysis integrated edge based technique for automatic vehicular license plate recognition framework,» *IET Intelligent Transport Systems*, vol. 14, n.º 7, págs. 712-723, 2020. DOI: 10.1049/iet-its.2019.0006. dirección: https://doi.org/10.1049/iet-its.2019.0006.

[7] C. Patel, D. Shah y A. Patel, «Automatic Number Plate Recognition System (ANPR): A Survey,» *International Journal of Computer Applications*, vol. 69 no.9, 2013.

[8] M. Zaizai, *License Plate Recognition YOLOv7 and CNN*, https://github.com/mrzaizai2k/License-Plate-Recognition-YOLOv7-and-CNN, Accedido: 22 oct. 2025, 2023.

[9] Y. Corochann, *MNIST training with Multi-Layer Perceptron*, https://corochann.com/mnist-training-with-multi-layer-perceptron-536/, Accessed: 2025-10-18, 2017.

[10] J. Shashirangana, H. Padmasiri, D. Meedeniya y C. Perera, «Automated License Plate Recognition: A Survey on Methods and Techniques,» *IEEE Access*, vol. 9, págs. 11 203-11 225, 2021. DOI: 10.1109/ACCESS.2020.3047929.

[11] R. Girshick, *Fast R-CNN*, 2015. arXiv: 1504.08083 [cs.CV]. dirección: https://arxiv.org/abs/1504.08083.

[12] S. Ren, K. He, R. Girshick y J. Sun, «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,» en *Advances in Neural Information Processing Systems*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama y R. Garnett, eds., vol. 28, Curran Associates, Inc., 2015. dirección: https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf.

[13] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, *You Only Look Once: Unified, Real-Time Object Detection*, 2016. arXiv: 1506.02640 [cs.CV]. dirección: https://arxiv.org/abs/1506.02640.

[14] A. Vina. «Using Ultralytics YOLO11 for Automatic Number Plate Recognition.» Accessed: 20 de noviembre de 2025. dirección: https://www.ultralytics.com/blog/using-ultralytics-yolo11-for-automatic-number-plate-recognition.

[15] R. Khanam y M. Hussain, *YOLOv11: An Overview of the Key Architectural Enhancements*, 2024. arXiv: 2410.17725 [cs.CV]. dirección: https://arxiv.org/abs/2410.17725.

[16] A. Vaswani et al., *Attention Is All You Need*, 2023. arXiv: 1706.03762 [cs.CL]. dirección: https://arxiv.org/abs/1706.03762.

[17] A. Hassani, S. Walton, N. Shah, A. Abuduweili, J. Li y H. Shi, «Escaping the Big Data Paradigm with Compact Transformers,» 2021. arXiv: 2104.05704 [cs.CV]. dirección: https://arxiv.org/abs/2104.05704.

[18] A. Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, 2021. arXiv: 2010.11929 [cs.CV]. dirección: https://arxiv.org/abs/2010.11929.

[19] M. Hardt y B. Recht, *Patterns, predictions, and actions: A story about machine learning*, 2021. arXiv: 2102.05242 [cs.LG]. dirección: https://arxiv.org/abs/2102.05242.

[20] Z. E. Vargoorani y C. Y. Suen, «License Plate Detection and Character Recognition Using Deep Learning and Font Evaluation,» *arXiv preprint arXiv:2412.12572*, 2024, Submitted 17 Dec 2024. dirección: https://doi.org/10.48550/arXiv.2412.12572.

[21] *Artículo 97: Códigos de las placas*, es, Sistema Costarricense de Información Jurídica (SCIJ), Poder Judicial de Costa Rica, Norma NRM 74140 / NRM 110458, 2013. dirección: https://pgrweb.go.cr/scij/Busqueda/Normativa/Normas/nrm_articulo.aspx?nValor1=1&nValor2=74140&nValor3=110458&param1=NRA.

[22] G. Jocher y J. Qiu, *Ultralytics YOLO11*, ver. 11.0.0, 2024. dirección: https://github.com/ultralytics/ultralytics.

[23] G. Jocher, J. Qiu y A. Chaurasia, *Ultralytics YOLO*, ver. 8.0.0, ene. de 2023. dirección: https://github.com/ultralytics/ultralytics.

[24] S. Zherzdev y A. Gruzdev, *LPRNet: License Plate Recognition via Deep Neural Networks*, 2018. arXiv: 1806.10447 [cs.CV]. dirección: https://arxiv.org/abs/1806.10447.

[25] L. Wu, S. Liu, Y. Xie, H. Lee y X. Duan, «A Single-Stage Multi-Style License Plate Recognition Method Based on Attention,» en *Proceedings of the 16th Asian Conference on Machine Learning*, V. Nguyen y H.-T. Lin, eds., ép. Proceedings of Machine Learning Research, vol. 260, PMLR, 2025, págs. 495-510. dirección: https://proceedings.mlr.press/v260/wu25a.html.